FPT UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



GROUP FINAL PROJECT
FAP201

Lecturer: Phạm Thế Vinh

Class: IC2007

Topic: IP Timer (AMBA APB)

Students:

Phùng Văn Tân - SE203222

Hoàng Trung Hiếu - SE203208

Hoàng Quốc Trí - SE203836

Lê Dương Hoàng Khang - SE204508

Trần Định Quốc - CE202008

Ho Chi Minh City, Ngày 4 Tháng 11 Năm 2026

LỜI CẢM ƠN

Đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Thầy Phạm Thế Vinh – giảng viên Trường Đại học FPT, người đã tận tình hướng dẫn, cung cấp tài liệu đặc tả và định hướng phương pháp thiết kế, kiểm thử vi mạch cho nhóm chúng em trong suốt học phần FAP201.

Trong 3 năm học tập và rèn luyện tại Trường Đại học FPT, em đã được truyền đạt những kiến thức nền tảng quý báu về lĩnh vực Thiết kế Vi mạch Bán dẫn: từ tư duy mô tả phần cứng bằng Verilog, chuẩn giao tiếp bus AMBA cho tới quy trình kiểm thử tự động bằng testbench và mô hình tham chiếu. Môi trường học tập năng động cùng những dự án thực hành theo nhóm đã giúp em rèn luyện cả kỹ năng chuyên môn lẫn kỹ năng phối hợp, tích hợp công việc của nhiều thành viên thành một hệ thống hoàn chỉnh.

Tiếp đến, em xin gửi lời tri ân đặc biệt đến Thầy Phạm Thế Vinh bộ môn Thiết Kế Vi Mạch, Bán Dẫn đã tạo điều kiện để nhóm thực hiện thành công báo cáo đồ án “Thiết kế và Mô phỏng IP Timer 64-bit giao tiếp AMBA APB”. Những kiến thức và trải nghiệm từ đồ án này là hành trang quan trọng để em tự tin bước vào kỳ thực tập và theo đuổi con đường Kỹ sư Thiết kế Vi mạch (Digital IC Design Engineer) trong tương lai. Em cũng xin cảm ơn các bạn trong nhóm đã cùng nhau hoàn thành từng khối thiết kế.

Cuối cùng, em xin gửi lời cảm ơn đến gia đình, người thân và bạn bè – những người đã luôn là hậu phương vững chắc, là chỗ dựa tinh thần giúp em vượt qua những thử thách trong học tập để hoàn thành tốt đồ án này.

Trân trọng,

Hoàng Quốc Trí

Thành phố Hồ Chí Minh, Ngày 4 Tháng 11 Năm 2026

Hoàng Quốc Trí
Khoa Vi Mạch, Bán Dẫn - Lớp IC2007

MỤC LỤC

Chương 1. GIỚI THIỆU TỔNG QUAN ĐỀ TÀI....9
1.1. Bối cảnh và nhu cầu thực tế....9
1.2. Mục tiêu và phương pháp thực hiện đề tài....10
1.2.1. Mục tiêu của đề tài....10
1.2.2. Phương pháp thực hiện....10
1.3. Bảng thông số kỹ thuật của hệ thống....11
1.4. Phân công công việc trong nhóm....11
Chương 2. CƠ SỞ LÝ THUYẾT....13
2.1. Giao thức bus AMBA APB....13
2.1.1. Khái niệm và đặc điểm của bus APB....13
2.1.2. Cấu trúc một giao dịch APB (SETUP – ACCESS – Wait state)....14
2.2. Bài toán chia tần số cho bộ định thời (Prescaler)....15
2.2.1. Nguyên lý bộ đếm định thời 64-bit....15
2.2.2. Giải pháp chia tần bằng xung cho phép đếm (Clock Enable)....15
2.3. Cơ chế ngắt RW1C và chế độ dừng gỡ lỗi (Debug Halt)....16
2.4. Phương pháp kiểm thử tự động với Golden Model....17
Chương 3. THIẾT KẾ PHẦN CỨNG....20
3.1. Sơ đồ khối tổng thể của hệ thống IP Timer....20
3.2. Thiết kế chi tiết module giao tiếp bus (apb_slave)....21
3.2.1. Module apb_slave và nguyên lý hoạt động....21
3.2.2. Sơ đồ máy trạng thái (FSM) tương đương của apb_slave....22
3.2.3. Cơ chế chèn chu kỳ chờ và phản hồi lỗi PSLVERR....23
3.3. Thiết kế chi tiết module thanh ghi (timer_register)....24
3.3.1. Module timer_register và nguyên lý hoạt động....24
3.3.2. Bản đồ thanh ghi (Register Map)....26
3.3.3. Ghi theo byte strobe (PSTRB)....27
3.3.4. Kiểm tra lỗi và bảo vệ cấu hình TCR....27
3.3.5. Thanh ghi trạng thái ngắt TISR và thanh ghi THCSR....29
3.4. Thiết kế chi tiết module chia tần (counter_divider)....29
3.4.1. Nguyên lý tạo xung cho phép đếm div_tick....29
3.4.2. Tính toán hệ số chia tần....29
3.4.3. Module counter_divider và nguyên lý hoạt động....30
3.5. Thiết kế chi tiết module lõi đếm (counter_core)....31
3.5.1. Module counter_core và nguyên lý hoạt động....31
3.5.2. Thứ tự ưu tiên cập nhật bộ đếm 64-bit....32
3.5.3. Bộ so sánh và tạo sự kiện ngắt....34
3.6. Module TOP (timer_top) và cơ chế ghép nối các khối....34
Chương 4. KẾT QUẢ KIỂM THỬ VÀ MÔ PHỎNG HỆ THỐNG....36
4.1. Môi trường kiểm thử tự động (Self-checking Testbench)....36
4.2. Kết quả mô phỏng trên phần mềm ModelSim....37
4.2.1. Kịch bản kiểm tra giá trị reset của các thanh ghi (TEST 1)....37
4.2.2. Kịch bản truy cập theo byte strobe và địa chỉ dự trữ (TEST 2)....38
4.2.3. Kịch bản cấu hình bộ chia tần và bộ đếm 64-bit (TEST 3 – TEST 4)....38
4.2.4. Kịch bản ngắt so sánh và xóa cờ RW1C (TEST 5)....39
4.2.5. Kịch bản dừng gỡ lỗi, chia tần và reset bất đồng bộ (TEST 6 – TEST 8)....39
4.2.6. Kịch bản kiểm thử khối thanh ghi (register_tb)....40
4.2.7. Kịch bản đối chiếu bộ đếm với Golden Model (counter_tb)....41

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN CỦA ĐỀ TÀI.....42
5.1. Kết luận.....42
5.2. Những khó khăn trong quá trình thiết kế.....42
5.3. Hướng phát triển trong tương lai.....43

DANH MỤC HÌNH

Hình 1.1: Sơ đồ vị trí của IP Timer trong hệ thống SoC giao tiếp qua bus AMBA APB.....9
Hình 2.1: Mô hình kết nối APB Master – APB Slave (IP Timer).....13
Hình 2.2: Sơ đồ máy trạng thái truyền dữ liệu của giao thức APB.....14
Hình 2.3: Giản đồ thời gian giao dịch ghi và đọc APB có chèn 1 chu kỳ chờ.....15
Hình 2.4: Bộ chia tần tạo xung cho phép đếm div_tick trong cùng một miền xung nhịp.....16
Hình 2.5: Giản đồ thời gian của xung div_tick với div_val = 2.....16
Hình 2.6: Ví dụ đoạn code về cơ chế RW1C của thanh ghi TISR.....17
Hình 2.7: Đường tạo ngắt tim_int và tín hiệu dừng gỡ lỗi halt_ack.....17
Hình 2.8: Sơ đồ môi trường kiểm thử (Testbench) sử dụng mô hình tham chiếu (Golden Model).....18
Hình 2.9: Sơ đồ khối tổng quát của môi trường kiểm thử (Testbench).....18
Hình 2.10: Đoạn code Scoreboard so sánh RTL với Golden Model ở mọi chu kỳ.....19
Hình 3.1: Sơ đồ khối kiến trúc tổng thể của hệ thống lõi IP Timer.....20
Hình 3.2: Khối APB_SLAVE.....21
Hình 3.3: Sơ đồ máy trạng thái (FSM) tương đương của khối apb_slave.....23
Hình 3.4: Đoạn code tạo chu kỳ chờ và các tín hiệu phản hồi của apb_slave.....24
Hình 3.5: Khối TIMER_REGISTER.....25
Hình 3.6: Cấu trúc trường bit của các thanh ghi TCR, TIER, TISR và THCSR.....26
Hình 3.7: Hàm merge_bytes gộp dữ liệu ghi theo byte strobe.....27
Hình 3.8: Lưu đồ xử lý lệnh ghi thanh ghi TCR.....28
Hình 3.9: Đoạn code kiểm tra lỗi và cập nhật thanh ghi TCR.....28
Hình 3.10: Khối COUNTER_DIVIDER.....30
Hình 3.11: Lưu đồ hoạt động của khối counter_divider.....31
Hình 3.12: Đoạn code của khối counter_divider.....31
Hình 3.13: Khối COUNTER_CORE.....32
Hình 3.14: Lưu đồ thứ tự ưu tiên cập nhật bộ đếm của khối counter_core.....33
Hình 3.15: Đoạn code cập nhật bộ đếm 64-bit của counter_core.....33
Hình 3.16: Khối TIMER_TOP.....34
Hình 3.17: Đoạn code tạo reg_wr_en_w và thanh ghi tdr_shadow trong timer_top.....35
Hình 4.1: Báo cáo kết quả kiểm tra tự động (Test Summary) của khối TIMER_TOP và TIMER_REGISTER.....37
Hình 4.2: Mô phỏng Waveform đọc giá trị reset của 8 thanh ghi qua bus APB.....37
Hình 4.3: Mô phỏng Waveform ghi thanh ghi TCMP0 với PSTRB = 0001, 0011, 1111.....38
Hình 4.4: Mô phỏng Waveform tín hiệu PSLVERR khi ghi TCR không hợp lệ và ghi TDR0/TDR1.....38
Hình 4.5: Mô phỏng Waveform tín hiệu tim_int khi bộ đếm bằng TCMP và khi xóa cờ bằng W1C.....39
Hình 4.6: Mô phỏng Waveform tín hiệu halt_ack, divider_tick và reset bất đồng bộ.....39
Hình 4.7: Mô phỏng Waveform khối thanh ghi T1 – T6: cấu hình TCR và tín hiệu reg_err....40
Hình 4.8: Mô phỏng Waveform khối thanh ghi T7 – T12: TDR, cờ ngắt TISR và THCSR.....40
Hình 4.9: Mô phỏng Waveform so sánh counter.v với GOLDEN_MODEL.v (div_val = 0 – 2).....41
Hình 4.10: Mô phỏng Waveform so sánh counter.v với GOLDEN_MODEL.v (div_val = 3 – 4).....41
Hình 5.1: Ví dụ IP Timer nhiều kênh so sánh kết nối với CPU RISC-V.....43
Hình 5.2: Ví dụ tạo xung PWM từ bộ đếm và giá trị so sánh.....43
Hình 5.3: Ví dụ luồng triển khai IP Timer lên board FPGA.....44

DANH MỤC BẢNG

Bảng 1.1: Thông số kỹ thuật cấu hình IP Timer.....11
Bảng 1.2: Phân công công việc trong nhóm.....12
Bảng 2.1: Các tín hiệu của giao thức AMBA APB và tên tương ứng trong thiết kế.....14
Bảng 2.2: So sánh hai phương pháp chia tần.....15
Bảng 3.1: Đặc tả khối apb_slave.....22
Bảng 3.2: Đặc tả khối timer_register.....25
Bảng 3.3: Bản đồ thanh ghi (Register Map) của IP Timer.....26
Bảng 3.4: Mô tả chi tiết các trường bit điều khiển.....27
Bảng 3.5: Các trường hợp ghi thanh ghi TCR và phản hồi của IP.....28
Bảng 3.6: Quan hệ giữa div_val, ngưỡng đếm và chu kỳ tick (sys_clk = 100 MHz).....29
Bảng 3.7: Đặc tả khối counter_divider.....30
Bảng 3.8: Đặc tả khối counter_core.....32
Bảng 3.9: Đặc tả khối timer_top.....35

DANH MỤC TỪ VIẾT TẮT

AHB	Advanced High-performance Bus
AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
ARM	Advanced RISC Machines
ASIC	Application-Specific Integrated Circuit
AXI	Advanced eXtensible Interface
CDC	Clock Domain Crossing
CPU	Central Processing Unit
DUT	Design Under Test
FF	Flip-Flop
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine
IC	Integrated Circuit
IP	Intellectual Property
LSB	Least Significant Bit
MSB	Most Significant Bit
PLIC	Platform-Level Interrupt Controller
PWM	Pulse Width Modulation
RISC	Reduced Instruction Set Computer
RO	Read Only
RTL	Register Transfer Level
RW	Read / Write
RW1C	Read / Write-1-to-Clear
SoC	System on Chip
STA	Static Timing Analysis
TB	Testbench
TCMP	Timer Compare Register
TCR	Timer Control Register
TDR	Timer Data Register
THCSR	Timer Halt Control and Status Register
TIER	Timer Interrupt Enable Register
TISR	Timer Interrupt Status Register
W1C	Write-1-to-Clear

TÓM TẮT BÀI BÁO CÁO

Bộ định thời (Timer) là khối ngoại vi nền tảng của mọi vi điều khiển và hệ thống trên chip (SoC), cung cấp nhịp thời gian, ngắt định kỳ và khả năng đo thời gian cho phần mềm. Để đáp ứng nhu cầu này, đồ án trình bày giải pháp thiết kế lõi IP Timer 64-bit giao tiếp với CPU thông qua bus AMBA APB.

Quá trình thiết kế phần cứng được thực hiện bằng ngôn ngữ Verilog theo cấu trúc module hóa, bao gồm khối giao tiếp APB (apb_slave) có chèn chu kỳ chờ và phản hồi lỗi, khối thanh ghi (timer_register) hỗ trợ ghi theo byte strobe và cờ ngắt RW1C, bộ chia tần (counter_divider) theo hệ số 2^div_val với cơ chế bảo vệ cấu hình, và lõi đếm (counter_core) 64-bit với bộ so sánh tạo ngắt và chế độ dừng gỡ lỗi (Debug Halt), được tích hợp trong module timer_top chạy trên một miền xung nhịp duy nhất. Hệ thống được kiểm chứng bằng bốn môi trường kiểm thử tự động (Self-Checking Testbench), trong đó khối bộ đếm được đối chiếu với mô hình tham chiếu (Golden Model) ở mọi chu kỳ, và dạng sóng được mô phỏng trên phần mềm ModelSim.

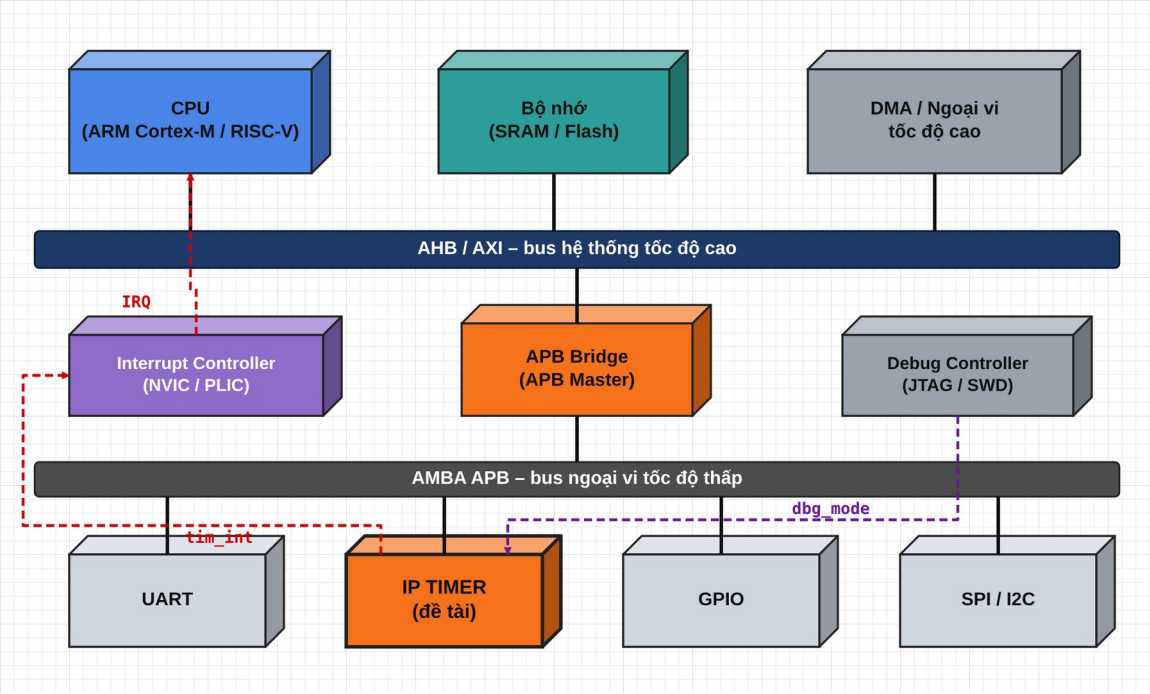
Kết quả mô phỏng trên ModelSim khẳng định mạch hoạt động đúng các kịch bản kiểm thử: testbench hệ thống đạt 45/45 điểm kiểm tra và khối thanh ghi đạt 87/87 điểm kiểm tra, đều 0 lỗi; dạng sóng của khối bộ đếm trùng khớp hoàn toàn với Golden Model. Thiết kế tạo tiền đề vững chắc để tích hợp vào SoC và tiến hành nạp thử nghiệm xuống board mạch FPGA.

Chương 1. **GIỚI THIỆU TỔNG QUAN ĐỀ TÀI**

1.1. **Bối cảnh và nhu cầu thực tế**

Trong mọi vi điều khiển (Microcontroller) và hệ thống trên chip (SoC/ASIC), bộ định thời (Timer) là một trong những khối ngoại vi không thể thiếu. Timer cung cấp "nhịp thời gian" cho toàn hệ thống: tạo ngắt định kỳ (system tick) cho hệ điều hành thời gian thực, đo khoảng thời gian giữa hai sự kiện, tạo độ trễ chính xác cho phần mềm hoặc giám sát quá thời gian (timeout) của các giao tiếp ngoại vi. CPU truy cập Timer giống như truy cập bộ nhớ: đọc/ghi các thanh ghi được ánh xạ địa chỉ thông qua bus hệ thống.

Trong kiến trúc SoC dựa trên chuẩn AMBA của ARM, các ngoại vi tốc độ thấp như Timer, UART, GPIO được gắn trên bus **APB (Advanced Peripheral Bus)** – một bus đơn giản, tiêu thụ ít năng lượng, nối với bus hệ thống tốc độ cao (AHB/AXI) thông qua cầu nối APB Bridge. Vị trí của IP Timer trong một SoC điển hình được thể hiện ở Hình 1.1.



Hình 1.1: Sơ đồ vị trí của IP Timer trong hệ thống SoC giao tiếp qua bus AMBA APB.

Tuy nhiên, một Timer 32-bit chạy ở tần số 100 MHz sẽ tràn sau khoảng 2^{32} / 100 MHz ≈ 42,9 giây, buộc phần mềm phải liên tục xử lý tràn số. Với bộ đếm 64-bit, thời gian tràn kéo dài tới khoảng 5.845 năm, đủ để dùng như một đồng hồ hệ thống suốt vòng đời sản phẩm. Bên cạnh đó, khi lập trình viên dừng CPU để gỡ lỗi (debug), Timer cũng phải có khả năng "đóng băng" để trạng thái hệ thống không bị sai lệch – đây là tính năng Debug Halt.

Xuất phát từ nhu cầu thực tiễn đó, đề tài được thực hiện nhằm thiết kế và kiểm thử một lõi IP Timer 64-bit hoàn chỉnh giao tiếp AMBA APB ở mức RTL, có bộ chia tần lập

trình được, ngắt so sánh, chế độ dừng gỡ lỗi và được kiểm chứng độ tin cậy thông qua quy trình mô phỏng tự động trên phần mềm ModelSim.

1.2. **Mục tiêu và phương pháp thực hiện đề tài**

1.2.1. **Mục tiêu của đề tài**

Đề tài hướng tới việc thiết kế thành công một lõi IP Timer có thể tích hợp trực tiếp vào SoC với các mục tiêu cụ thể như sau:

- **Giao tiếp chuẩn AMBA APB:** Hỗ trợ đầy đủ các tín hiệu `tim_pstrb` (ghi theo byte), `tim_pready` (chèn 1 chu kỳ chờ) và `tim_pslverr` (báo lỗi), địa chỉ 12-bit, dữ liệu 32-bit.
- **Bộ đếm 64-bit đọc/ghi được:** Bộ đếm lên 64-bit truy cập qua hai thanh ghi 32-bit TDR0/TDR1, cho phép phần mềm ghi từng byte.
- **Bộ chia tần (Prescaler) có bảo vệ cấu hình:** Chia tần số đếm theo 2^div_val (div_val = 0..7), từ chối các giá trị 8..15 và mọi thay đổi div_en/div_val khi timer đang chạy.
- **Ngắt so sánh 64-bit:** So sánh bộ đếm với TCMP1:TCMP0; cờ ngắt TISR.int_st được xóa bằng cơ chế ghi 1 để xóa (RW1C) và có thể che ngắt bằng TIER.int_en.
- **Chế độ Debug Halt:** Đóng băng bộ chia tần và bộ đếm khi hệ thống ở chế độ gỡ lỗi và phần mềm cho phép dừng.
- **Đảm bảo tiêu chuẩn RTL:** Một miền xung nhịp duy nhất, reset bất đồng bộ tích cực mức thấp, tách biệt logic tổ hợp và logic tuần tự, không sinh mạch chốt (Latch-free).

1.2.2. **Phương pháp thực hiện**

Do tính chất của đề tài tập trung vào khâu Front-end (RTL Design & Verification) và không triển khai trực tiếp lên phần cứng FPGA, phương pháp thực hiện được tiếp cận theo quy trình kiểm thử vi mạch:

- **Ngôn ngữ mô tả phần cứng:** Sử dụng Verilog-2001 để mô tả vi mạch ở mức RTL theo cấu trúc module hóa; mỗi thành viên phụ trách một khối chức năng.
- **Quy trình kiểm thử tự động (Verification):** Thay vì quan sát dạng sóng bằng mắt thường một cách thủ công, đề tài xây dựng bốn môi trường kiểm thử tự kiểm tra (Self-Checking Testbench): ba testbench cho từng khối và một testbench cho toàn hệ thống.
- **Sử dụng Golden Model:** Mô hình tham chiếu `GOLDEN_MODEL.py` (Python) của đề bài được chuyển thành module Verilog `GOLDEN_MODEL.v`

để chạy song song với RTL; Scoreboard so sánh hai bên ở mọi chu kỳ xung nhịp trên trình mô phỏng ModelSim.

- **Mô phỏng trên ModelSim:** Thiết kế được biên dịch và mô phỏng trên ModelSim – Intel FPGA Starter Edition 2020.1 bằng các script `run_*.do`; dạng sóng thực tế được đưa vào Chương 4.

1.3. **Bảng thông số kỹ thuật của hệ thống**

Cấu hình mặc định của lõi IP Timer được sử dụng để thiết kế và kiểm thử trong đề tài được thể hiện qua Bảng 1.1.

Thông số (Parameter)	Giá trị mặc định	Mô tả chi tiết
Giao tiếp bus	AMBA APB slave	Hỗ trợ PSTRB, PREADY, PSLVERR.
ADDR_W	12 bit	Sử dụng các offset 0x000 – 0x01C; hai bit thấp được bỏ qua khi giải mã.
Độ rộng dữ liệu	32 bit	Ghi theo byte bằng tim_pstrb[3:0].
Chu kỳ chờ	1 chu kỳ	Mỗi giao dịch APB mất 3 chu kỳ sys_clk (SETUP + 2 chu kỳ ACCESS).
Bộ đếm	64 bit, đếm lên	Đọc/ghi qua TDR0 (bit 31:0) và TDR1 (bit 63:32).
Bộ chia tần	2^div_val	div_val = 0..7 (chia 1 đến chia 128); giá trị 8..15 bị từ chối.
Giá trị so sánh	64 bit	TCMP1:TCMP0, giá trị reset 0xFFFF_FFFF_FFFF_FFFF.
Ngắt	tim_int	tim_int = int_st & int_en; TISR xóa bằng RW1C.
Debug Halt	halt_ack	halt_ack = dbg_mode & halt_req, đóng băng bộ đếm.
Clock / Reset	sys_clk / sys_rst_n	Một miền clock; reset bất đồng bộ, tích cực mức thấp.
CLK mô phỏng	100 MHz	Chu kỳ 10 ns trong các testbench.

Bảng 1.1: Thông số kỹ thuật cấu hình IP Timer.

1.4. **Phân công công việc trong nhóm**

Đề tài được thực hiện bởi nhóm 5 thành viên. Mỗi khối RTL được giao cho một thành viên chịu trách nhiệm thiết kế; thành viên phụ trách tích hợp hệ thống xây dựng module timer_top, các testbench, mô hình tham chiếu và thực hiện mô phỏng trên ModelSim (Bảng 1.2).

STT	Họ và tên	MSSV	Nhiệm vụ	Mã nguồn phụ trách
1	Phùng Văn Tân	SE203222	Timer Divider	counter_divider (counter.v)
2	Hoàng Trung Hiếu	SE203208	APB Slave Interface	apb_slave (app_slave.v)
3	Hoàng Quốc Trí	SE203836	System Integration and Verification	timer_top.v, GOLDEN_MODEL.v, 4 testbench, script ModelSim
4	Lê Dương Hoàng Khang	SE204508	Timer Core	counter_core, counter (counter.v)
5	Trần Định Quốc	CE202008	Register Bank	timer_register (register.v)

Bảng 1.2: Phân công công việc trong nhóm.

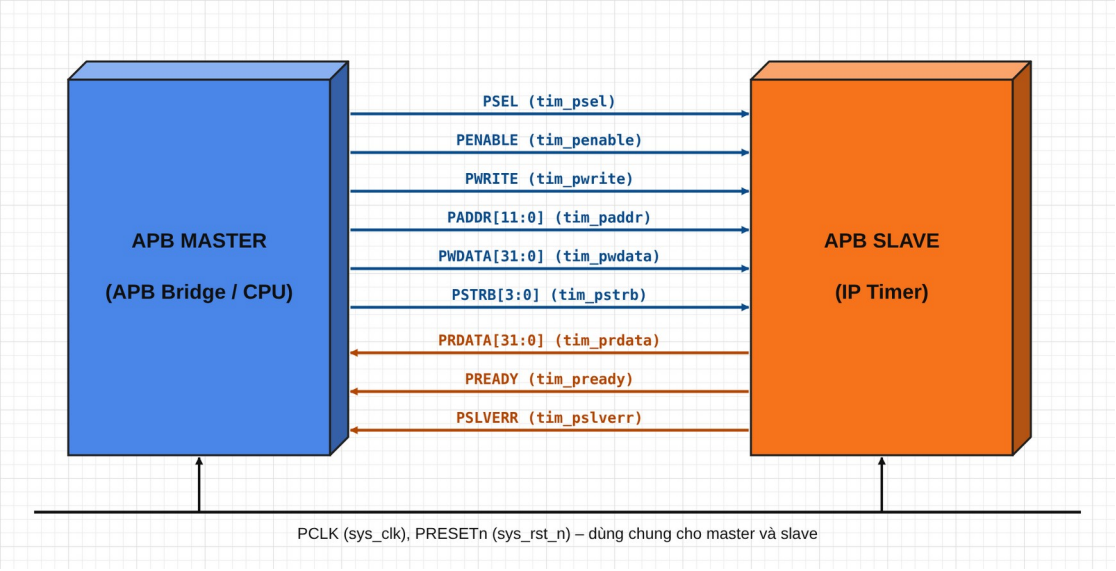
Chương 2. **CƠ SỞ LÝ THUYẾT**

2.1. **Giao thức bus AMBA APB**

2.1.1. **Khái niệm và đặc điểm của bus APB**

AMBA (Advanced Microcontroller Bus Architecture) là bộ chuẩn bus trên chip do ARM phát triển, gồm AXI, AHB và APB. Trong đó **APB (Advanced Peripheral Bus)** được tối ưu cho các ngoại vi tốc độ thấp: không có cơ chế đường ống (non-pipelined), mọi tín hiệu được lấy mẫu tại sườn lên của PCLK và mỗi giao dịch kéo dài tối thiểu hai chu kỳ. Phiên bản APB4 bổ sung tín hiệu PSTRB cho phép ghi theo từng byte.

Kiến trúc APB hoạt động theo mô hình **Master – Slave**: APB Bridge đóng vai trò master, phát địa chỉ, chiều truyền và dữ liệu; IP Timer đóng vai trò slave, phản hồi bằng PREADY, PRDATA và PSLVERR. Toàn bộ giao dịch đồng bộ với một xung nhịp chung, vì vậy slave không cần xử lý bài toán lệch miền xung nhịp (Hình 2.1).



Hình 2.1: Mô hình kết nối APB Master – APB Slave (IP Timer).

Các tín hiệu của giao thức APB và tên tương ứng trong thiết kế IP Timer được liệt kê trong Bảng 2.1.

Tín hiệu APB	Tên trong thiết kế	Nguồn	Mô tả
PCLK	sys_clk	Hệ thống	Xung nhịp, mọi tín hiệu lấy mẫu ở sườn lên.
PRESETn	sys_rst_n	Hệ thống	Reset tích cực mức thấp.
PSELx	tim_psel	Master	Chọn slave, bắt đầu giao dịch.
PENABLE	tim_penable	Master	Báo hiệu pha ACCESS.
PWRITE	tim_pwrite	Master	1 = ghi, 0 = đọc.
PADDR	tim_paddr[11:0]	Master	Địa chỉ thanh ghi (offset byte).
PWDATA	tim_pwdata[31:0]	Master	Dữ liệu ghi.
PSTRB	tim_pstrb[3:0]	Master	Chọn byte của PWDATA được ghi.

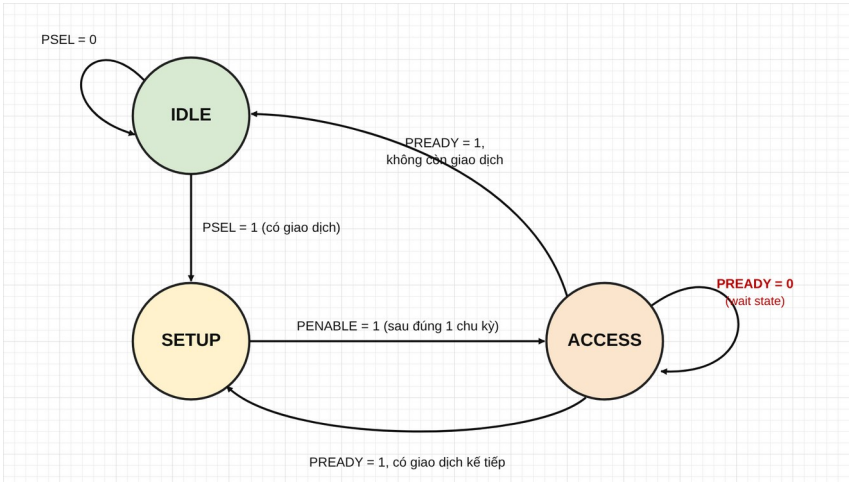
Tín hiệu APB	Tên trong thiết kế	Nguồn	Mô tả
PRDATA	tim_prdata[31:0]	Slave	Dữ liệu đọc, master lấy mẫu khi PREADY = 1.
PREADY	tim_pready	Slave	Slave hoàn tất giao dịch; = 0 để chèn chu kỳ chờ.
PSLVERR	tim_pslverr	Slave	Báo lỗi giao dịch, chỉ có nghĩa ở chu kỳ hoàn tất.

Bảng 2.1: Các tín hiệu của giao thức AMBA APB và tên tương ứng trong thiết kế.

2.1.2. **Cấu trúc một giao dịch APB (SETUP – ACCESS – Wait state)**

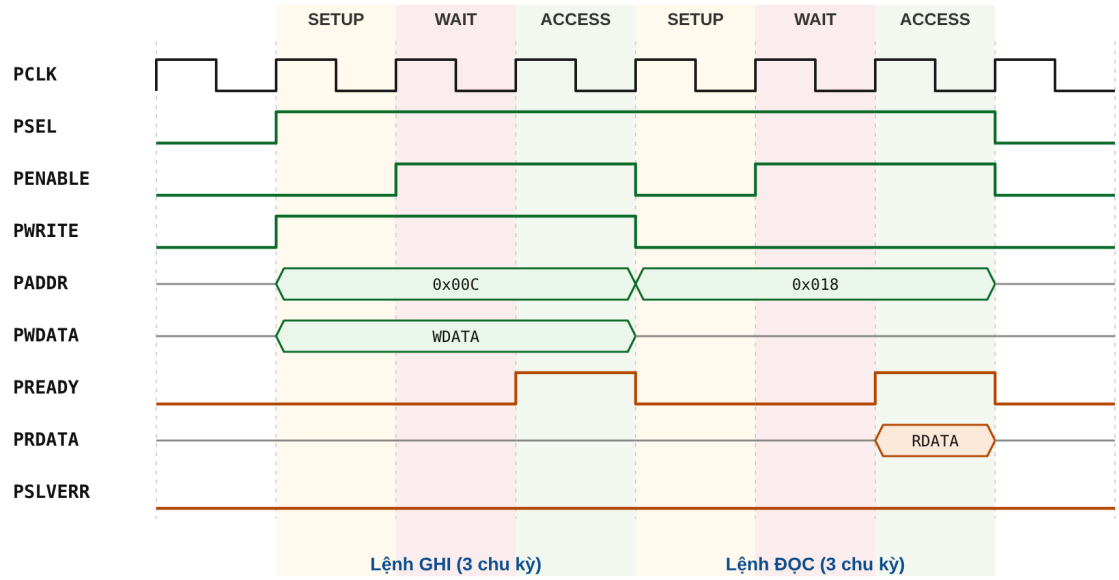
Dữ liệu trên bus APB không được truyền liên tục mà được đóng gói thành từng giao dịch (transfer). Hoạt động của bus được mô tả bằng máy trạng thái ba trạng thái (Hình 2.2):

- **Trạng thái nghỉ (IDLE):** Trạng thái mặc định, không có giao dịch (PSEL = 0).
- **Pha thiết lập (SETUP):** Master đặt PSEL = 1, PENABLE = 0 cùng địa chỉ, chiều và dữ liệu. Trạng thái này chỉ kéo dài đúng một chu kỳ.
- **Pha truy cập (ACCESS):** Master đặt PENABLE = 1 và giữ ổn định mọi tín hiệu. Nếu PREADY = 0, bus ở lại ACCESS (wait state); khi PREADY = 1, giao dịch hoàn tất và bus chuyển về IDLE hoặc sang SETUP nếu có giao dịch kế tiếp.



Hình 2.2: Sơ đồ máy trạng thái truyền dữ liệu của giao thức APB.

Hình 2.3 minh họa một lệnh ghi theo sau là một lệnh đọc khi slave chèn 1 chu kỳ chờ. Ở chu kỳ SETUP, master đưa địa chỉ và dữ liệu; ở chu kỳ ACCESS đầu tiên slave giữ PREADY = 0; tới chu kỳ ACCESS thứ hai PREADY = 1, dữ liệu ghi được chốt (lệnh ghi) hoặc PRDATA được master lấy mẫu (lệnh đọc). Như vậy mỗi giao dịch của IP Timer mất đúng 3 chu kỳ sys_clk. Tín hiệu PSLVERR chỉ được xem xét ở chu kỳ hoàn tất, khi PSEL, PENABLE và PREADY đều bằng 1.



Hình 2.3: Giản đồ thời gian giao dịch ghi và đọc APB có chèn 1 chu kỳ chờ.

2.2. **Bài toán chia tần số cho bộ định thời (Prescaler)**

2.2.1. **Nguyên lý bộ đếm định thời 64-bit**

Lõi của Timer là một thanh ghi đếm lên: mỗi khi có điều kiện cho phép, giá trị tăng thêm 1. Phần mềm đọc giá trị đếm để biết thời gian đã trôi qua, hoặc nạp một giá trị so sánh để được báo ngắt khi bộ đếm chạm tới. Vì bus dữ liệu APB chỉ rộng 32 bit, bộ đếm 64-bit được ánh xạ thành hai thanh ghi: **TDR0** chứa 32 bit thấp và **TDR1** chứa 32 bit cao; tương tự giá trị so sánh 64-bit được chia thành **TCMP0** và **TCMP1**.

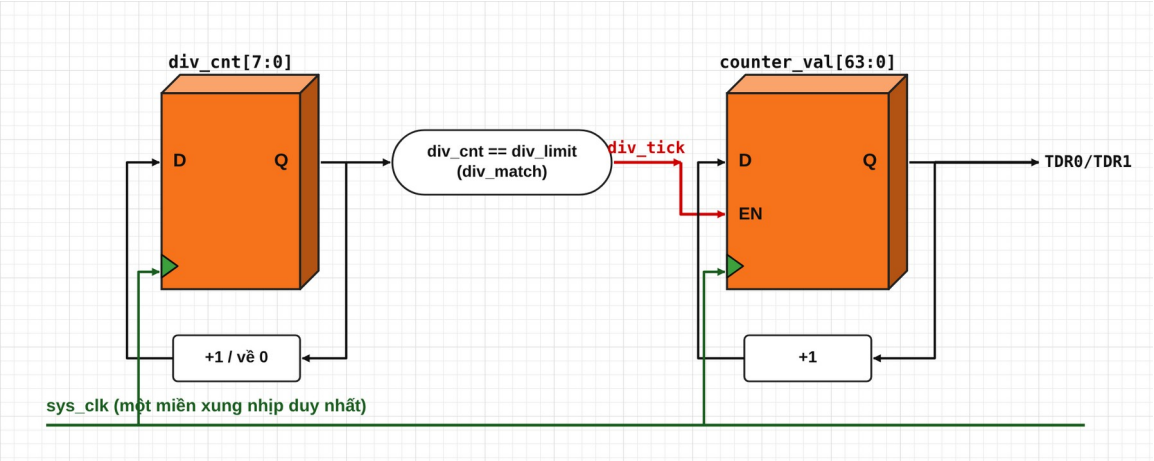
Trong nhiều ứng dụng, tần số sys_clk quá nhanh so với nhịp thời gian cần đo. Bộ chia tần (Prescaler) giúp bộ đếm chỉ tăng sau mỗi 2^div_val chu kỳ, qua đó mở rộng dải thời gian đo mà không cần tăng thêm số bit của bộ đếm.

2.2.2. **Giải pháp chia tần bằng xung cho phép đếm (Clock Enable)**

Có hai cách làm bộ đếm chạy chậm hơn sys_clk: tạo ra một xung clock mới có tần số thấp hơn, hoặc giữ nguyên clock và chỉ cho phép đếm ở một số chu kỳ nhất định. Cách thứ nhất sinh ra miền clock mới, gây lệch pha (clock skew), phải xử lý giao tiếp giữa các miền clock (CDC) và làm phức tạp phân tích định thời tĩnh. Vì vậy thiết kế sử dụng cách thứ hai: khối chia tần chỉ tạo ra xung **div_tick** rộng đúng 1 chu kỳ làm tín hiệu cho phép đếm, toàn bộ IP vẫn chạy chung một miền clock sys_clk (Bảng 2.2, Hình 2.4).

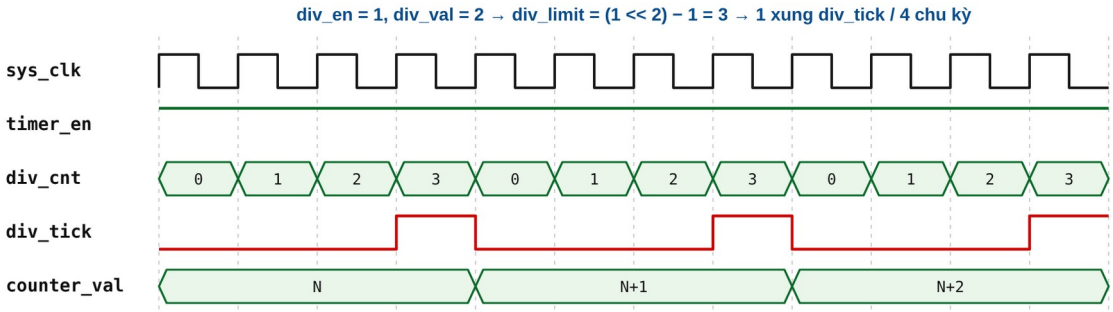
Tiêu chí	Tạo clock mới	Xung cho phép đếm (div_tick)
Số miền clock	Nhiều miền clock	Một miền clock sys_clk
Đồng bộ dữ liệu (CDC)	Bắt buộc phải xử lý	Không cần
Phân tích định thời	Phức tạp (generated clock)	Đơn giản
Thay đổi hệ số chia	Dễ sinh xung clock lỗi (glitch)	An toàn, chỉ đổi ngưỡng đếm

Bảng 2.2: So sánh hai phương pháp chia tần.



Hình 2.4: Bộ chia tần tạo xung cho phép đếm div_tick trong cùng một miền xung nhịp.

Với div_en = 1, bộ đếm div_cnt đếm từ 0 tới ngưỡng div_limit = 2^div_val − 1 rồi quay về 0; xung div_tick xuất hiện đúng ở chu kỳ div_cnt chạm ngưỡng. Hình 2.5 minh họa trường hợp div_val = 2: cứ 4 chu kỳ sys_clk có một xung div_tick và bộ đếm 64-bit tăng 1.



Hình 2.5: Giản đồ thời gian của xung div_tick với div_val = 2.

2.3. **Cơ chế ngắt RW1C và chế độ dừng gỡ lỗi (Debug Halt)**

Ngắt là cách IP Timer chủ động báo cho CPU biết một sự kiện thời gian đã xảy ra, thay vì để phần mềm phải liên tục thăm dò. Thiết kế áp dụng các cơ chế sau:

- **Cờ ngắt được chốt (latched):** Khi bộ đếm bằng giá trị so sánh, phần cứng bật cờ int_st. Cờ giữ nguyên cho tới khi phần mềm chủ động xóa, kể cả khi bộ đếm đã vượt qua giá trị so sánh.
- **Che ngắt (Interrupt Mask):** Ngõ ra gửi tới CPU là tim_int = int_st & int_en. Phần mềm có thể tắt ngắt (int_en = 0) nhưng vẫn thăm dò (polling) được cờ int_st.
- **Ghi 1 để xóa (RW1C – Read / Write-1-to-Clear):** Ghi 1 vào bit int_st của TISR sẽ xóa cờ, ghi 0 không có tác dụng. Phần mềm xóa chính xác cờ đã xử lý mà không cần thao tác đọc – sửa – ghi.

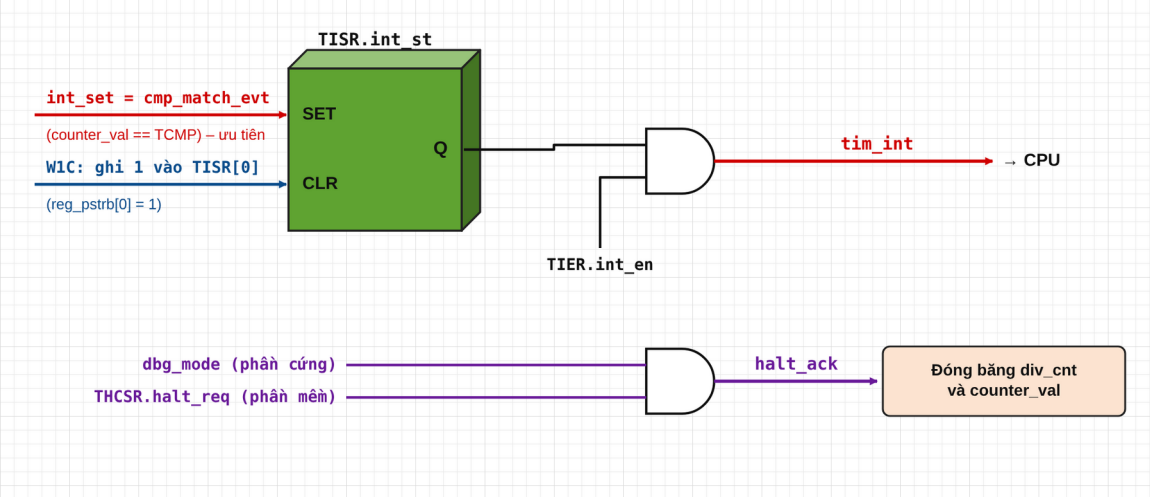
- **Ưu tiên sự kiện phần cứng:** Khi lệnh xóa của phần mềm và sự kiện bật cờ xảy ra trong cùng một chu kỳ, câu lệnh set đứng sau trong khối always nên cờ vẫn được giữ, ngắt không bị bỏ sót (Hình 2.6).

register.v

```
// Co che RW1C cua thanh ghi TISR
if (wr_tisr && reg_pstrb[0] && reg_wdata[0])
    r_int_st <= 1'b0;      // SW ghi 1 -> xoa co
if (int_set)
    r_int_st <= 1'b1;      // HW set (uu tien hon)
assign tim_int = r_int_st && r_int_en;
```

Hình 2.6: Ví dụ đoạn code về cơ chế RW1C của thanh ghi TISR.

Khi lập trình viên dừng CPU tại một điểm ngắt (breakpoint), nếu Timer vẫn tiếp tục đếm thì sau khi chạy tiếp, phần mềm sẽ thấy thời gian "nhảy vọt" và có thể phát sinh ngắt không mong muốn. Vì vậy IP Timer hỗ trợ chế độ **Debug Halt** với cơ chế bắt tay hai tín hiệu: phần mềm đặt bit halt_req trong thanh ghi THCSR để cho phép dừng, còn tín hiệu phần cứng dbg_mode báo hệ thống đang ở chế độ gỡ lỗi. Chỉ khi cả hai cùng bằng 1, Timer mới dừng và báo lại bằng bit chỉ đọc halt_ack = dbg_mode & halt_req. Trong thời gian dừng, cả bộ đếm chia tần lẫn bộ đếm 64-bit đều giữ nguyên giá trị (Hình 2.7).



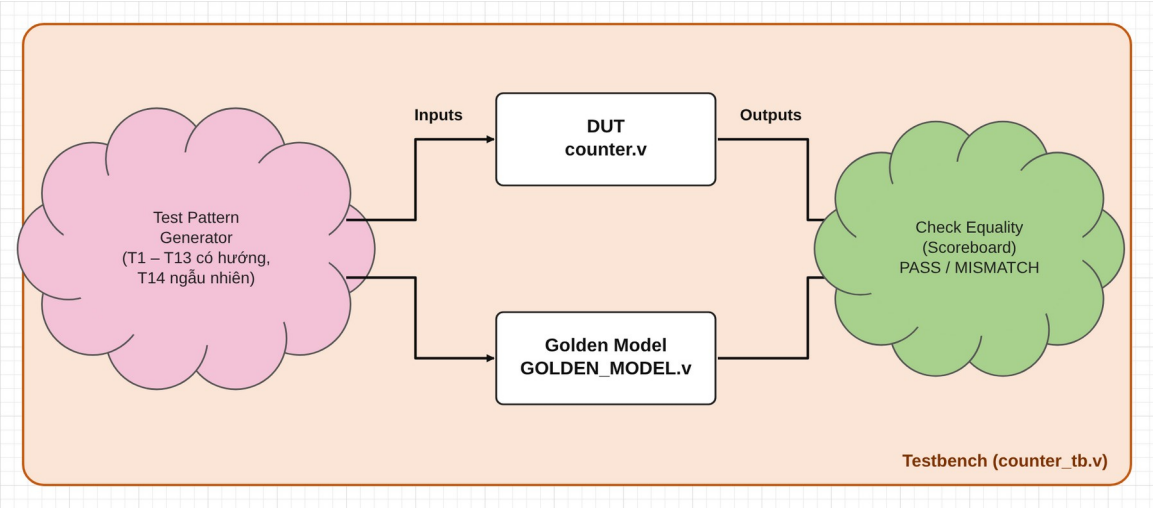
Hình 2.7: Đường tạo ngắt tim_int và tín hiệu dừng gỡ lỗi halt_ack.

2.4. **Phương pháp kiểm thử tự động với Golden Model**

Trong quy trình thiết kế vi mạch chuyên nghiệp (IC Verification), việc kiểm tra tính đúng đắn của thiết kế bằng cách nhìn bằng mắt thường các dạng sóng mô phỏng là không khả thi với số lượng trường hợp lớn. Do đó, phương pháp **Self-Checking Testbench** kết hợp **Golden Model** được áp dụng.

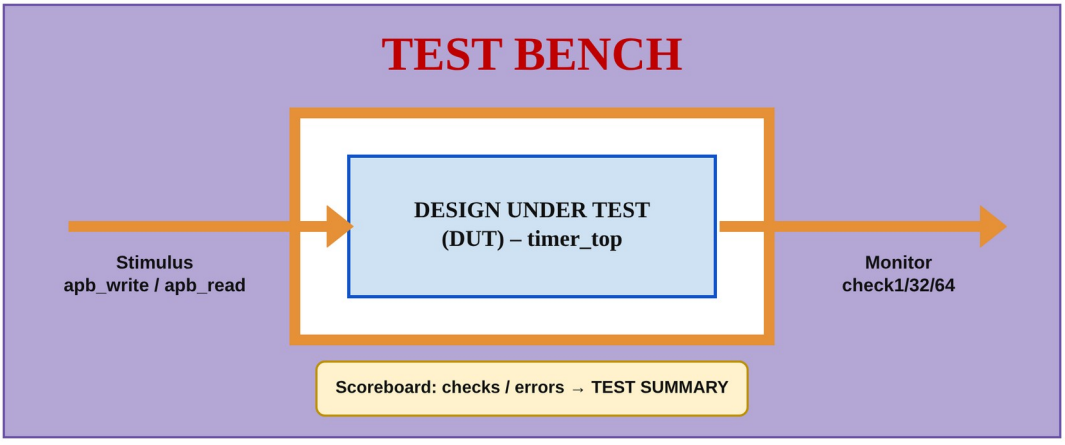
- **Golden Model:** Là mô hình hành vi bậc cao mô phỏng lại chính xác các hành vi kỳ vọng của phần cứng. Đề bài cung cấp mô hình `GOLDEN_MODEL.py` (Python) cho khối counter; vì ModelSim chỉ biên

dịch Verilog/SystemVerilog, nhóm viết lại mô hình này thành module Verilog `counter_golden_model` (tệp `GOLDEN_MODEL.v`) theo đúng cấu trúc step_clk() / eval_combinational() của bản Python. Golden Model chạy song song với DUT (Design Under Test) trong testbench (Hình 2.8).



Hình 2.8: Sơ đồ môi trường kiểm thử (Testbench) sử dụng mô hình tham chiếu (Golden Model).

- **Testbench tích hợp Stimulus, Monitor & Scoreboard:** Testbench đóng vai CPU, phát các giao dịch APB theo đúng giao thức (Stimulus), đồng thời có các khối giám sát liên tục thu thập đầu ra (Monitor). Khối Scoreboard so sánh tự động (Auto-comparison) giữa kết quả xuất ra từ RTL và đáp án kỳ vọng hoặc Golden Model; mọi sai khác được báo FAIL kèm thời điểm, cuối mô phỏng in ra dòng TEST SUMMARY mà không cần kỹ sư dò sóng thủ công (Hình 2.9, Hình 2.10).



Hình 2.9: Sơ đồ khối tổng quát của môi trường kiểm thử (Testbench).

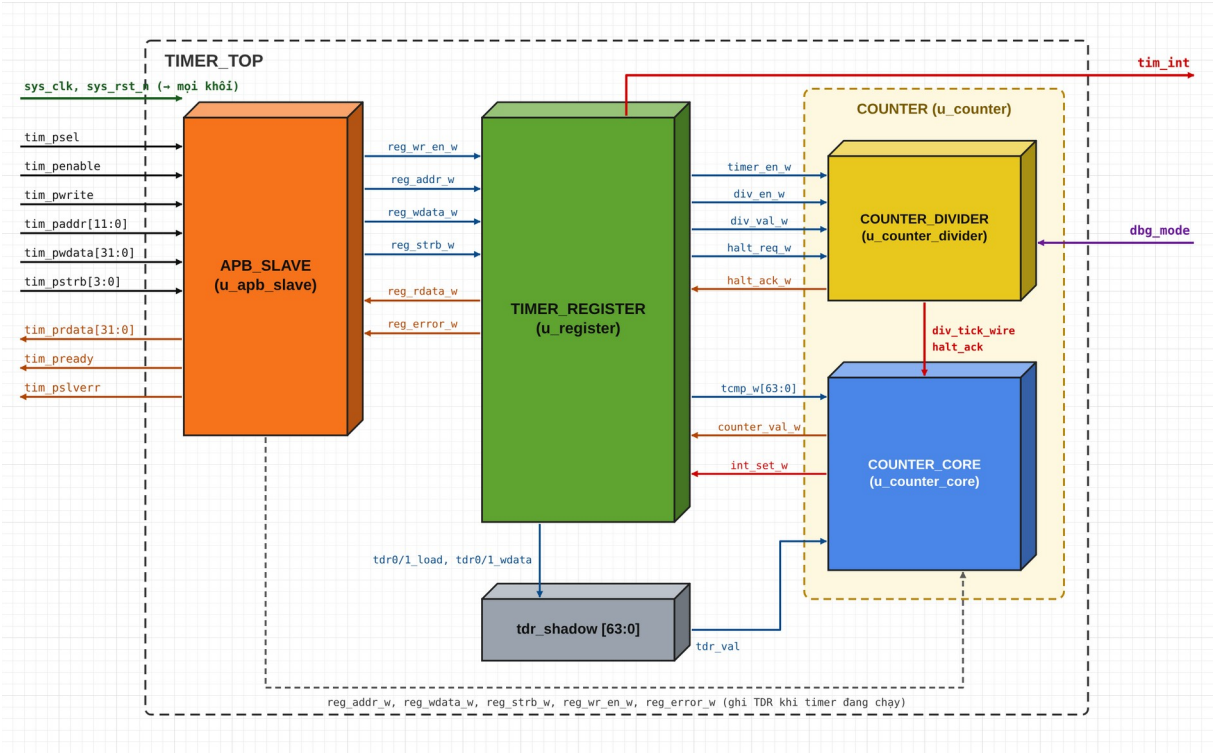
counter_tb.v

```
// counter_tb: Scoreboard so sanh RTL voi GOLDEN_MODEL moi chu ky
always @(posedge sys_clk) begin
    #2;
    sb_cycles = sb_cycles + 1;
    if ((counter_val   !== gm_counter_val)   ||
        (cmp_match_evt !== gm_cmp_match_evt) ||
        (halt_ack      !== gm_halt_ack)      ||
        (dut_div_cnt   !== gm_div_cnt)       ||
        (dut_div_tick  !== gm_div_tick))
        sb_errors = sb_errors + 1;    // + in thong bao MISMATCH
end
```

Hình 2.10: Đoạn code Scoreboard so sánh RTL với Golden Model ở mọi chu kỳ.

Chương 3. **THIẾT KẾ PHẦN CỨNG**

3.1. **Sơ đồ khối tổng thể của hệ thống IP Timer**



Hình 3.1: Sơ đồ khối kiến trúc tổng thể của hệ thống lõi IP Timer.

Hệ thống lõi IP Timer được thiết kế theo cấu trúc module hóa (Modular Design) nhằm tăng tính tái sử dụng và dễ dàng cô lập lỗi trong quá trình kiểm thử. Kiến trúc tổng thể bao gồm các khối chức năng được gói gọn bên trong module tầng cao nhất `timer_top`:

- `apb_slave`: Khối giao tiếp bus, nhận giao dịch APB, chèn 1 chu kỳ chờ, phát yêu cầu đọc/ghi đã hoàn tất sang khối thanh ghi và trả về dữ liệu đọc cùng phản hồi lỗi.
- `timer_register`: Khối thanh ghi TCR, TDR0/1, TCMP0/1, TIER, TISR, THCSR; giải mã địa chỉ, ghi theo byte strobe, kiểm tra lỗi cấu hình TCR và tạo ngõ ra ngắt tim_int.
- `counter_divider`: Khối chia tần, tạo xung cho phép đếm div_tick và tín hiệu halt_ack.
- `counter_core`: Khối lõi đếm, chứa bộ đếm 64-bit và bộ so sánh tạo sự kiện ngắt cmp_match_evt. Hai khối counter_divider và counter_core được đóng gói trong module `counter`.
- `tdr_shadow`: Thanh ghi 64-bit trong timer_top lưu giá trị TDR do phần mềm lập trình; bộ đếm lấy giá trị này khi timer đang tắt.

Luồng dữ liệu chính: lệnh ghi từ CPU đi qua apb_slave → timer_register; các trường cấu hình (timer_en, div_en, div_val, tcmp, halt_req) được đưa tới counter; ngược lại, giá trị đếm counter_val, sự kiện so sánh int_set và halt_ack được đưa về timer_register để phần mềm đọc. Toàn bộ IP dùng chung sys_clk và sys_rst_n.

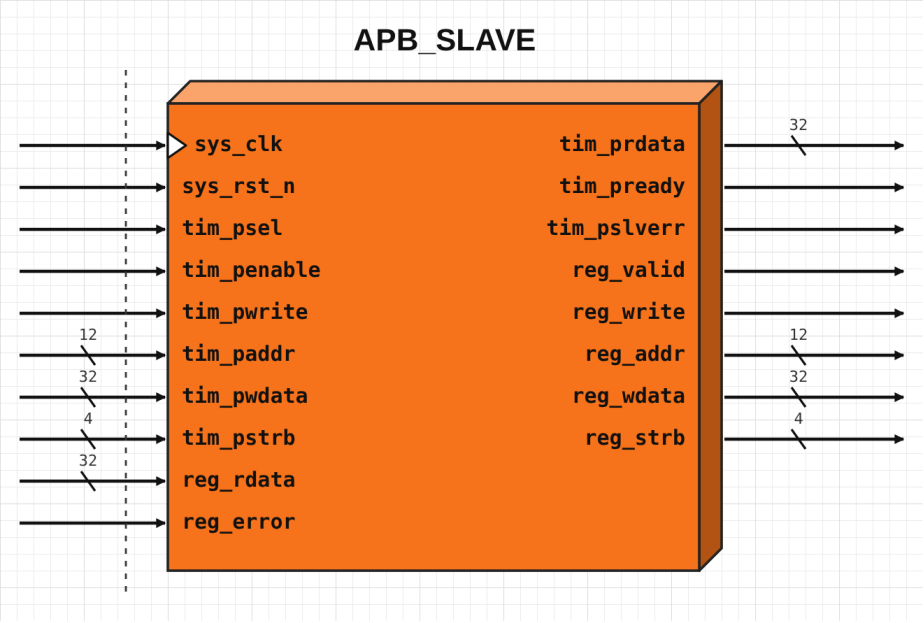
3.2. **Thiết kế chi tiết module giao tiếp bus (apb_slave)**

3.2.1. **Module apb_slave và nguyên lý hoạt động**

Khối apb_slave (tệp app_slave.v) là "cửa ngõ" giữa CPU và IP Timer. Nhiệm vụ của khối này là nhận một giao dịch APB, xác định đúng thời điểm giao dịch hoàn tất và chuyển yêu cầu đó sang khối thanh ghi đúng một lần.

Nguyên lý hoạt động cơ bản:

- Khối apb_slave luôn theo dõi điều kiện pha ACCESS: `apb_access = tim_psel && tim_penable`.
- Ở chu kỳ ACCESS đầu tiên, khối giữ tim_pready = 0 (chèn 1 chu kỳ chờ) và dùng flip-flop access_seen để ghi nhớ rằng chu kỳ này đã trôi qua.
- Ở chu kỳ ACCESS thứ hai, tim_pready = 1: giao dịch hoàn tất, reg_valid = 1 báo cho khối thanh ghi chốt lệnh ghi hoặc trả dữ liệu đọc qua tim_prdata.
- Nếu khối thanh ghi báo lỗi (reg_error = 1) trong một lệnh ghi, tim_pslverr được bật lên mức cao đúng ở chu kỳ hoàn tất.



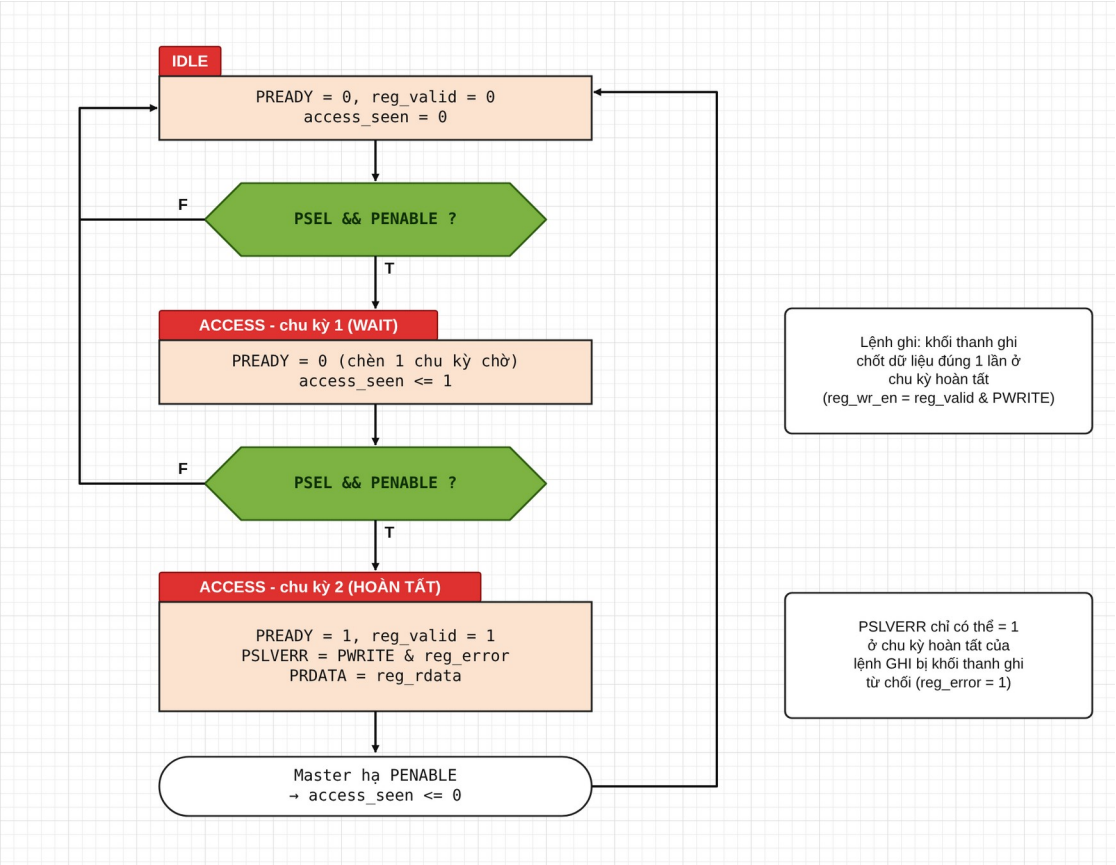
Hình 3.2: Khối APB_SLAVE.

Tên cổng	Chiều	Độ rộng	Chức năng
sys_clk	input	1 bit	Xung nhịp hệ thống (System clock).
sys_rst_n	input	1 bit	Reset bất đồng bộ tích cực mức thấp (Active-low reset).
tim_psel, tim_penable	input	1 bit	Chọn slave và báo hiệu pha ACCESS của APB.
tim_pwrite	input	1 bit	Chiều giao dịch: 1 = ghi, 0 = đọc.
tim_paddr	input	12 bits	Địa chỉ thanh ghi (offset byte).
tim_pwdata	input	32 bits	Dữ liệu ghi.
tim_pstrb	input	4 bits	Byte strobe của lệnh ghi.
tim_prdata	output	32 bits	Dữ liệu đọc, nối trực tiếp từ reg_rdata.
tim_pready	output	1 bit	Báo hoàn tất giao dịch (= apb_access & access_seen).
tim_pslverr	output	1 bit	Báo lỗi lệnh ghi (= tim_pready & tim_pwrite & reg_error).
reg_valid, reg_write	output	1 bit	Yêu cầu đã hoàn tất (= tim_pready) và chiều ghi.
reg_addr / reg_wdata / reg_strb	output	12 / 32 / 4 bits	Địa chỉ, dữ liệu, byte strobe chuyển sang khối thanh ghi.
reg_rdata	input	32 bits	Dữ liệu đọc từ khối thanh ghi.
reg_error	input	1 bit	Lỗi ghi do khối thanh ghi phát hiện.

Bảng 3.1: Đặc tả khối apb_slave.

3.2.2. **Sơ đồ máy trạng thái (FSM) tương đương của apb_slave**

Khối apb_slave không cần một thanh ghi trạng thái nhiều bit: chỉ một flip-flop access_seen kết hợp với điều kiện apb_access là đủ để phân biệt ba trạng thái IDLE, ACCESS – chu kỳ 1 (WAIT) và ACCESS – chu kỳ 2 (hoàn tất). Hình 3.3 trình bày máy trạng thái tương đương của khối:



Hình 3.3: Sơ đồ máy trạng thái (FSM) tương đương của khối apb_slave.

- **Tại trạng thái IDLE:** Khi không có giao dịch hoặc đang ở pha SETUP (PENABLE = 0), access_seen bị xóa về 0 và tim_pready = 0.
- **Tại ACCESS – chu kỳ 1:** apb_access = 1 nhưng access_seen vẫn bằng 0, nên tim_pready = 0 – đây chính là chu kỳ chờ. Tại sườn lên kế tiếp access_seen được nạp 1.
- **Tại ACCESS – chu kỳ 2:** tim_pready = 1, reg_valid = 1; khối thanh ghi chốt lệnh ghi (reg_wr_en = reg_valid & reg_write) hoặc trả dữ liệu đọc. Sau đó master hạ PENABLE, access_seen trở về 0 và khối quay về IDLE.

3.2.3. **Cơ chế chèn chu kỳ chờ và phản hồi lỗi PSLVERR**

Toàn bộ khối được mô tả bằng một khối tuần tự duy nhất và các phép gán tổ hợp (Hình 3.4). Vì tim_pslverr được tạo từ tim_pready, tim_pwrite và reg_error, tín hiệu lỗi chỉ có thể bằng 1 ở chu kỳ hoàn tất của một lệnh **ghi**; lệnh đọc luôn hoàn tất với tim_pslverr = 0, đúng với đặc tả. Mỗi lệnh ghi APB tác động vào khối thanh ghi đúng một lần vì reg_valid chỉ bằng 1 ở chu kỳ hoàn tất.

app_slave.v

```
// apb_slave: chen 1 chu ky cho (wait state)
assign apb_access = tim_psel && tim_penable;
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n)       access_seen <= 1'b0;
    else if (!apb_access) access_seen <= 1'b0;
    else                  access_seen <= 1'b1;
end
assign tim_pready  = apb_access && access_seen;
assign tim_pslverr = tim_pready && tim_pwrite && reg_error;
assign tim_prdata  = reg_rdata;
assign reg_valid   = tim_pready;
```

Hình 3.4: Đoạn code tạo chu kỳ chờ và các tín hiệu phản hồi của apb_slave.

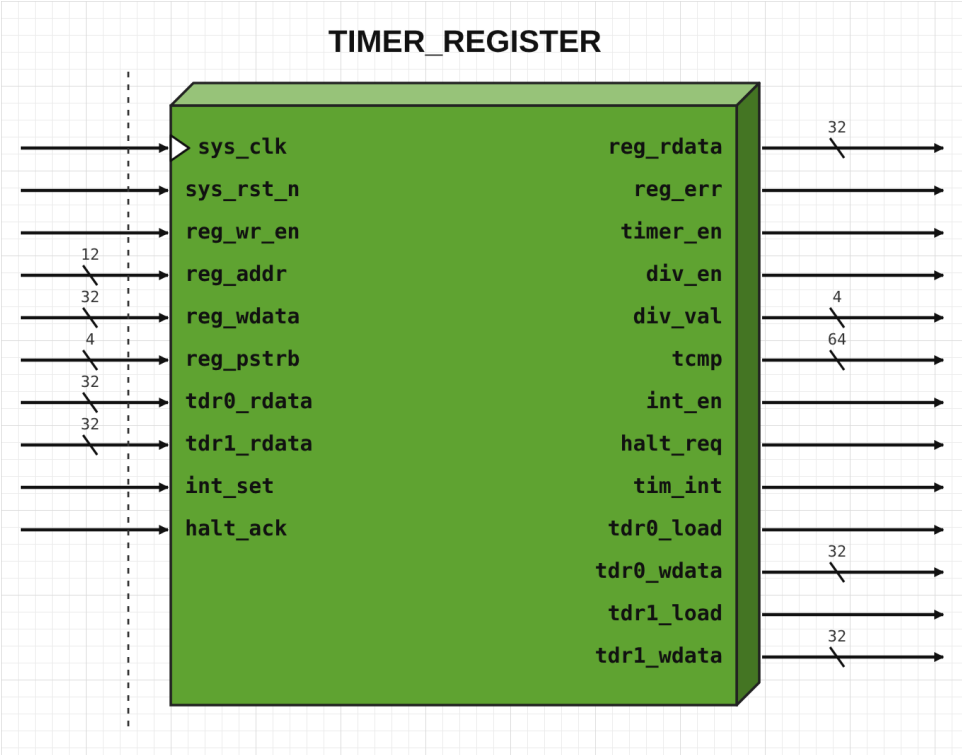
3.3. **Thiết kế chi tiết module thanh ghi (timer_register)**

3.3.1. **Module timer_register và nguyên lý hoạt động**

Khối timer_register (tệp register.v) đóng vai trò “bảng điều khiển” của IP Timer: lưu toàn bộ cấu hình do phần mềm lập trình và cung cấp trạng thái của bộ đếm cho CPU đọc.

Nguyên lý hoạt động cơ bản:

- Địa chỉ được căn theo word (`addr_aligned = {reg_addr[11:2], 2'b00}`) rồi giải mã thành các tín hiệu chọn sel_tcr … sel_thcsr.
- Khi reg_wr_en = 1, dữ liệu được gộp theo byte strobe và ghi vào thanh ghi được chọn; lệnh ghi TCR được kiểm tra lỗi trước khi cập nhật.
- Dữ liệu đọc được chọn bằng một khối case tổ hợp: TDR0/TDR1 lấy trực tiếp giá trị bộ đếm hiện tại, bit halt_ack lấy từ khối counter.
- Lệnh ghi TDR0/TDR1 được chuyển thành xung tdr0_load/tdr1_load kèm dữ liệu đã gộp byte để timer_top cập nhật tdr_shadow.



Hình 3.5: Khối TIMER_REGISTER.

Tên cổng	Chiều	Độ rộng	Chức năng
sys_clk, sys_rst_n	input	1 bit	Xung nhịp và reset bất đồng bộ tích cực mức thấp.
reg_wr_en	input	1 bit	Lệnh ghi APB đã hoàn tất.
reg_addr	input	12 bits	Địa chỉ thanh ghi.
reg_wdata / reg_pstrb	input	32 / 4 bits	Dữ liệu ghi và byte strobe.
reg_rdata	output	32 bits	Dữ liệu đọc theo địa chỉ (tổ hợp).
reg_err	output	1 bit	Lỗi ghi TCR (div_val 8..15 hoặc thay đổi khi đang chạy).
timer_en, div_en, div_val	output	1 / 1 / 4 bits	Các trường của TCR.
tcmp	output	64 bits	Giá trị so sánh {TCMP1, TCMP0}.
int_en, halt_req	output	1 bit	TIER.int_en và THCSR.halt_req.
tim_int	output	1 bit	Ngõ ra ngắt: r_int_st & r_int_en.
tdr0_load / tdr1_load	output	1 bit	Xung ghi TDR0 / TDR1.
tdr0_wdata / tdr1_wdata	output	32 bits	Dữ liệu TDR đã gộp theo byte strobe.
tdr0_rdata / tdr1_rdata	input	32 bits	Giá trị bộ đếm hiện tại (đọc TDR).
int_set	input	1 bit	Sự kiện so sánh từ khối counter.
halt_ack	input	1 bit	Trạng thái dừng gỡ lỗi từ khối counter.

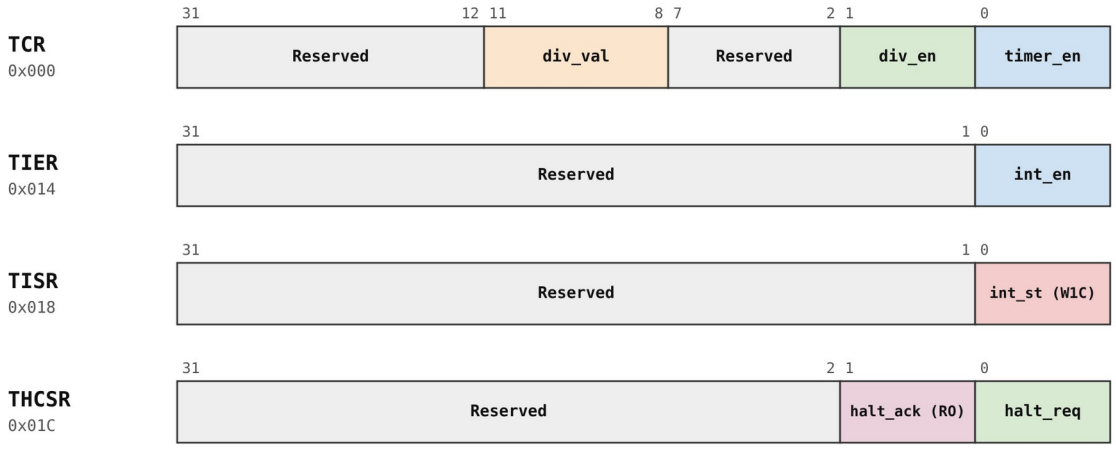
Bảng 3.2: Đặc tả khối timer_register.

3.3.2. **Bản đồ thanh ghi (Register Map)**

IP Timer có 8 thanh ghi 32-bit đặt liên tiếp từ offset 0x000 đến 0x01C (Bảng 3.3). Hai bit thấp của địa chỉ được bỏ qua (truy cập theo word). Mọi địa chỉ khác là vùng dự trữ (reserved): đọc trả về 0, ghi bị bỏ qua và không báo lỗi. Cấu trúc trường bit của các thanh ghi điều khiển được thể hiện ở Hình 3.6 và Bảng 3.4.

Offset	Tên	Truy cập	Giá trị reset	Mô tả
0x000	TCR	RW	0x0000_0000	Timer Control Register: timer_en, div_en, div_val.
0x004	TDR0	RW	0x0000_0000	Timer Data Register 0: bộ đếm bit [31:0].
0x008	TDR1	RW	0x0000_0000	Timer Data Register 1: bộ đếm bit [63:32].
0x00C	TCMP0	RW	0xFFFF_FFFF	Timer Compare 0: giá trị so sánh bit [31:0].
0x010	TCMP1	RW	0xFFFF_FFFF	Timer Compare 1: giá trị so sánh bit [63:32].
0x014	TIER	RW	0x0000_0000	Timer Interrupt Enable: bit int_en.
0x018	TISR	RW1C	0x0000_0000	Timer Interrupt Status: bit int_st.
0x01C	THCSR	RW / RO	0x0000_0000	Timer Halt Control/Status: halt_req, halt_ack.

Bảng 3.3: Bản đồ thanh ghi (Register Map) của IP Timer.



Hình 3.6: Cấu trúc trường bit của các thanh ghi TCR, TIER, TISR và THCSR.

Thanh ghi	Bit	Trường	Truy cập	Mô tả
TCR	[0]	timer_en	RW	Bật bộ đếm. Khi = 0, bộ đếm lấy giá trị TDR đã lập trình.
TCR	[1]	div_en	RW	0: đếm mỗi chu kỳ sys_clk; 1: đếm theo bộ chia tần.
TCR	[11:8]	div_val	RW	Hệ số chia 2^div_val; 0..7 hợp lệ, 8..15 bị từ chối.
TIER	[0]	int_en	RW	Cho phép đưa int_st ra ngõ tim_int.
TISR	[0]	int_st	RW1C	Cờ ngắt; phần cứng set khi counter = TCMP, ghi 1 để xóa.
THCSR	[0]	halt_req	RW	Phần mềm cho phép dừng khi gỡ lỗi.
THCSR	[1]	halt_ack	RO	Báo Timer đang dừng: dbg_mode & halt_req.

Bảng 3.4: Mô tả chi tiết các trường bit điều khiển.

3.3.3. **Ghi theo byte strobe (PSTRB)**

Hàm `merge_bytes(old, new, strobe)` tạo giá trị mới bằng cách lấy từng byte của dữ liệu ghi nếu bit strobe tương ứng bằng 1, ngược lại giữ byte cũ (Hình 3.7). Hàm này được dùng cho TCMP0/TCMP1, TCR, TIER, THCSR và cho dữ liệu ghi TDR (gộp với giá trị bộ đếm hiện tại). Lệnh ghi TISR chỉ có tác dụng khi pstrb[0] = 1, vì cờ int_st nằm ở byte 0.

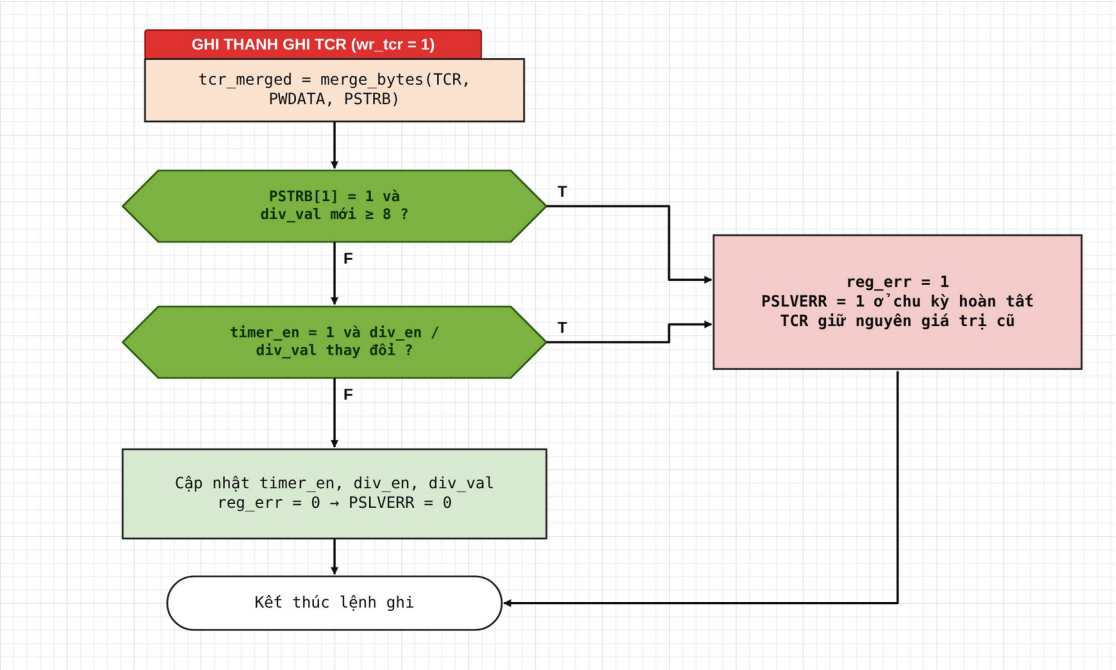
register.v

```
// Ghi theo byte strobe (PSTRB)
function [31:0] merge_bytes;
    input [31:0] old_value, new_value;
    input [3:0]  byte_strobe;
    merge_bytes = {
        byte_strobe[3] ? new_value[31:24] : old_value[31:24],
        byte_strobe[2] ? new_value[23:16] : old_value[23:16],
        byte_strobe[1] ? new_value[15:8]  : old_value[15:8],
        byte_strobe[0] ? new_value[7:0]   : old_value[7:0] };
endfunction
```

Hình 3.7: Hàm merge_bytes gộp dữ liệu ghi theo byte strobe.

3.3.4. **Kiểm tra lỗi và bảo vệ cấu hình TCR**

Giá trị TCR sau khi gộp byte (tcr_merged) được kiểm tra trước khi cập nhật (Hình 3.8). Lệnh ghi bị coi là lỗi trong hai trường hợp: ghi byte 1 với div_val mới thuộc 8..15 (bit div_val[3] = 1), hoặc timer đang chạy (r_timer_en = 1) mà giá trị mới của div_en hay div_val khác giá trị hiện tại. Khi có lỗi, toàn bộ lệnh ghi TCR bị bỏ qua và tim_pslverr = 1 ở chu kỳ hoàn tất.



Hình 3.8: Lưu đồ xử lý lệnh ghi thanh ghi TCR.

register.v

```
// Kiem tra loi khi ghi TCR
assign err_prohibited_divider   = wr_tcr && reg_pstrb[1] && next_div_val[3];
assign err_change_while_enabled = wr_tcr && r_timer_en &&
       ((next_div_en != r_div_en) || (next_div_val != r_div_val));
assign reg_err = err_prohibited_divider || err_change_while_enabled;

if (wr_tcr && !reg_err) begin       // chi cap nhat khi khong loi
    r_timer_en <= next_timer_en;
    r_div_en   <= next_div_en;
    r_div_val  <= next_div_val;
end
```

Hình 3.9: Đoạn code kiểm tra lỗi và cập nhật thanh ghi TCR.

Bảng 3.5 tổng hợp các trường hợp ghi TCR và phản hồi tương ứng của IP.

Trạng thái	Lệnh ghi TCR	tim_pslverr	Kết quả
timer_en = 0	div_val mới = 0..7	0	Cập nhật timer_en, div_en, div_val.
Bất kỳ	Ghi byte 1 với div_val = 8..15	1	Bỏ qua, TCR giữ nguyên.
timer_en = 1	Đổi div_en hoặc div_val	1	Bỏ qua, TCR giữ nguyên.
timer_en = 1	Giữ nguyên div_en, div_val; đổi timer_en	0	Cập nhật timer_en (dừng timer).
Bất kỳ	Địa chỉ reserved	0	Không thay đổi, đọc lại bằng 0.

Bảng 3.5: Các trường hợp ghi thanh ghi TCR và phản hồi của IP.

3.3.5. **Thanh ghi trạng thái ngắt TISR và thanh ghi THCSR**

Cờ r_int_st được xóa khi phần mềm ghi 1 vào bit 0 của TISR với pstrb[0] = 1, và được set khi tín hiệu int_set (sự kiện so sánh từ counter) bằng 1. Trong khối always, câu lệnh set đứng sau câu lệnh xóa nên sự kiện phần cứng được ưu tiên khi hai việc xảy ra cùng chu kỳ (Hình 2.6). Ngõ ra ngắt được tạo tổ hợp: `tim_int = r_int_st && r_int_en`.

Thanh ghi THCSR gồm bit halt_req (RW) do phần mềm điều khiển và bit halt_ack (RO) phản ánh trạng thái dừng thực tế từ khối counter; lệnh ghi vào halt_ack bị bỏ qua.

3.4. **Thiết kế chi tiết module chia tần (counter_divider)**

3.4.1. **Nguyên lý tạo xung cho phép đếm div_tick**

Vì IP Timer không tạo ra clock mới, khối counter_divider (tệp counter.v) phải tạo ra một xung cho phép (Clock Enable) **div_tick** rộng đúng 1 chu kỳ sys_clk. Nhờ xung này, bộ đếm 64-bit chỉ tăng ở những chu kỳ được cho phép, trong khi toàn bộ thiết kế vẫn đồng bộ với một xung nhịp duy nhất. Khi div_en = 0, div_tick bằng 1 ở mọi chu kỳ (không chia tần).

3.4.2. **Tính toán hệ số chia tần**

Khối chia tần được thiết kế dưới dạng bộ đếm tuần hoàn (Mod-N Counter) 8-bit. Ngưỡng đếm được tính bằng công thức thay vì gán cứng (Hardcode) một giá trị cụ thể:

$$div_limit = (1 << div_val) - 1 \qquad f_count = f_sys_clk / 2\hat{}div_val$$

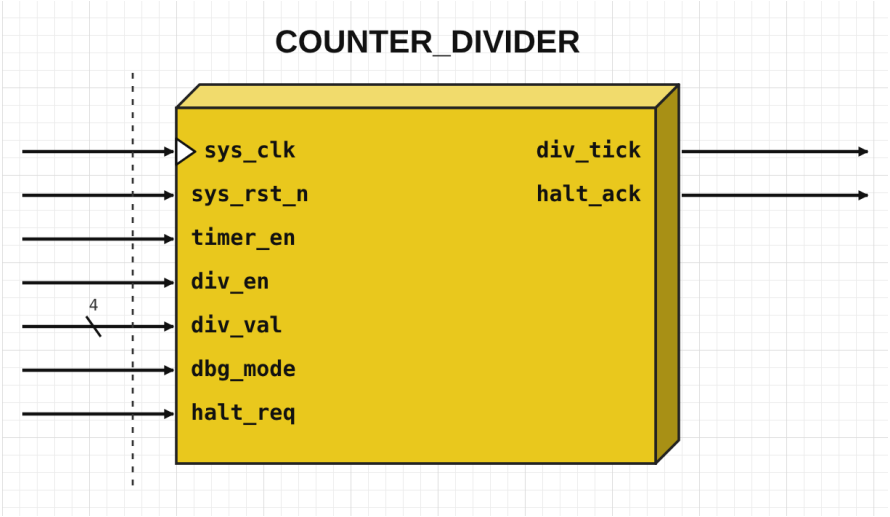
Ví dụ với sys_clk = 100 MHz và div_val = 7 (giá trị lớn nhất được phép), bộ đếm div_cnt đếm từ 0 đến 127, khi đạt 127 sẽ phát một xung div_tick rồi quay về 0; bộ đếm 64-bit tăng sau mỗi 128 chu kỳ (1,28 µs), tương ứng tần số đếm 781,25 kHz. Quan hệ đầy đủ được liệt kê trong Bảng 3.6.

div_val	div_limit	Hệ số chia	Chu kỳ tick (100 MHz)
0	0	1	10 ns
1	1	2	20 ns
2	3	4	40 ns
3	7	8	80 ns
4	15	16	160 ns
5	31	32	320 ns
6	63	64	640 ns
7	127	128	1,28 µs
8 – 15	—	Cấm	Ghi TCR bị từ chối (PSLVERR = 1)

Bảng 3.6: Quan hệ giữa div_val, ngưỡng đếm và chu kỳ tick (sys_clk = 100 MHz).

3.4.3. **Module counter_divider và nguyên lý hoạt động**

Khối counter_divider đóng vai trò như “nhịp tim” của bộ đếm. Bộ đếm div_cnt tăng ở mỗi sườn lên của sys_clk, được xóa về 0 khi chạm ngưỡng hoặc khi timer_en = 0, và giữ nguyên giá trị khi hệ thống đang dừng gỡ lỗi (is_halted = dbg_mode && halt_req). Tín hiệu halt_ack được lấy trực tiếp từ is_halted (Hình 3.10, Bảng 3.7).

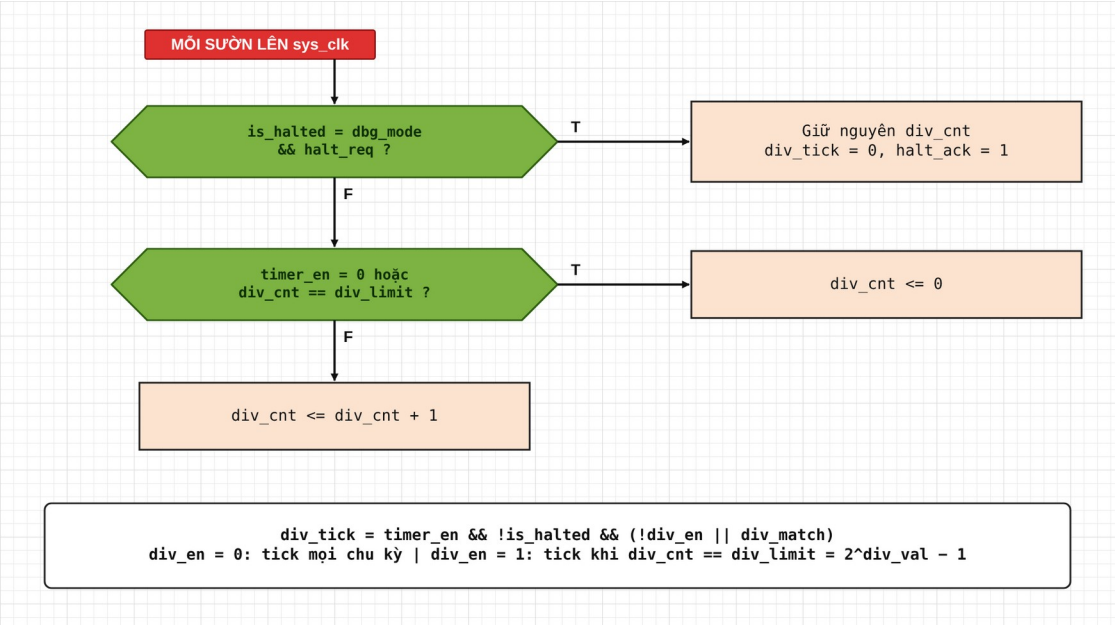


Hình 3.10: Khối COUNTER_DIVIDER.

Tên cổng	Chiều	Độ rộng	Chức năng
sys_clk	input	1 bit	Xung nhịp hệ thống (System clock).
sys_rst_n	input	1 bit	Reset bất đồng bộ, tích cực mức thấp.
timer_en	input	1 bit	Cho phép timer; = 0 thì xóa div_cnt.
div_en	input	1 bit	Cho phép chia tần.
div_val	input	4 bits	Hệ số chia: tick sau mỗi 2^div_val chu kỳ.
dbg_mode	input	1 bit	Hệ thống đang ở chế độ gỡ lỗi.
halt_req	input	1 bit	Yêu cầu dừng từ THCSR.
halt_ack	output	1 bit	dbg_mode & halt_req.
div_tick	output	1 bit	Xung cho phép đếm rộng 1 chu kỳ (không tạo clock mới).

Bảng 3.7: Đặc tả khối counter_divider.

Thuật toán hoạt động tại mỗi sườn lên của sys_clk được trình bày trong lưu đồ Hình 3.11 và mã nguồn Hình 3.12:



Hình 3.11: Lưu đồ hoạt động của khối counter_divider.

counter.v

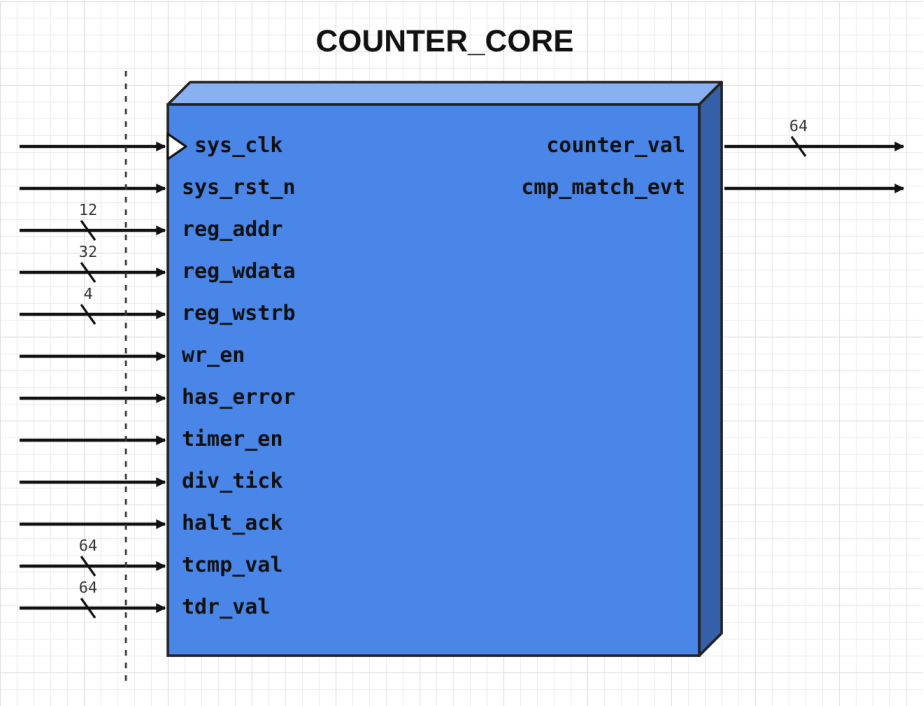
```
// counter_divider: xung cho phep dem div_tick
assign is_halted = dbg_mode && halt_req;
assign halt_ack  = is_halted;
assign div_limit = (8'd1 << div_val) - 8'd1;
assign div_match = (div_cnt == div_limit);
always @(posedge sys_clk or negedge sys_rst_n)
    if (!sys_rst_n)              div_cnt <= 8'd0;
    else if (!is_halted)
        if (div_match || !timer_en) div_cnt <= 8'd0;
        else                         div_cnt <= div_cnt + 8'd1;
assign div_tick = timer_en && !is_halted && (!div_en || div_match);
```

Hình 3.12: Đoạn code của khối counter_divider.

3.5. **Thiết kế chi tiết module lõi đếm (counter_core)**

3.5.1. **Module counter_core và nguyên lý hoạt động**

Khối counter_core (tệp counter.v) chứa bộ đếm 64-bit counter_val và bộ so sánh. Module counter bao ngoài khởi tạo hai instance u_counter_divider và u_counter_core, nối chúng qua dây div_tick_wire (Hình 3.13, Bảng 3.8).



Hình 3.13: Khối COUNTER_CORE.

Tên cổng	Chiều	Độ rộng	Chức năng
sys_clk, sys_rst_n	input	1 bit	Xung nhịp và reset bất đồng bộ tích cực mức thấp.
reg_addr	input	12 bits	Địa chỉ ghi (0x004: TDR0, 0x008: TDR1).
reg_wdata / reg_wstrb	input	32 / 4 bits	Dữ liệu và byte strobe ghi TDR.
wr_en, has_error	input	1 bit	Lệnh ghi hoàn tất, lỗi ghi.
timer_en	input	1 bit	= 0: bộ đếm lấy giá trị tdr_val.
div_tick	input	1 bit	Xung cho phép đếm từ counter_divider.
halt_ack	input	1 bit	Dừng đếm khi gỡ lỗi.
tcmp_val	input	64 bits	Giá trị so sánh.
tdr_val	input	64 bits	Giá trị TDR đã lập trình (tdr_shadow).
counter_val	output	64 bits	Giá trị bộ đếm (đọc qua TDR0/TDR1).
cmp_match_evt	output	1 bit	Mức 1 khi counter_val == tcmp_val.

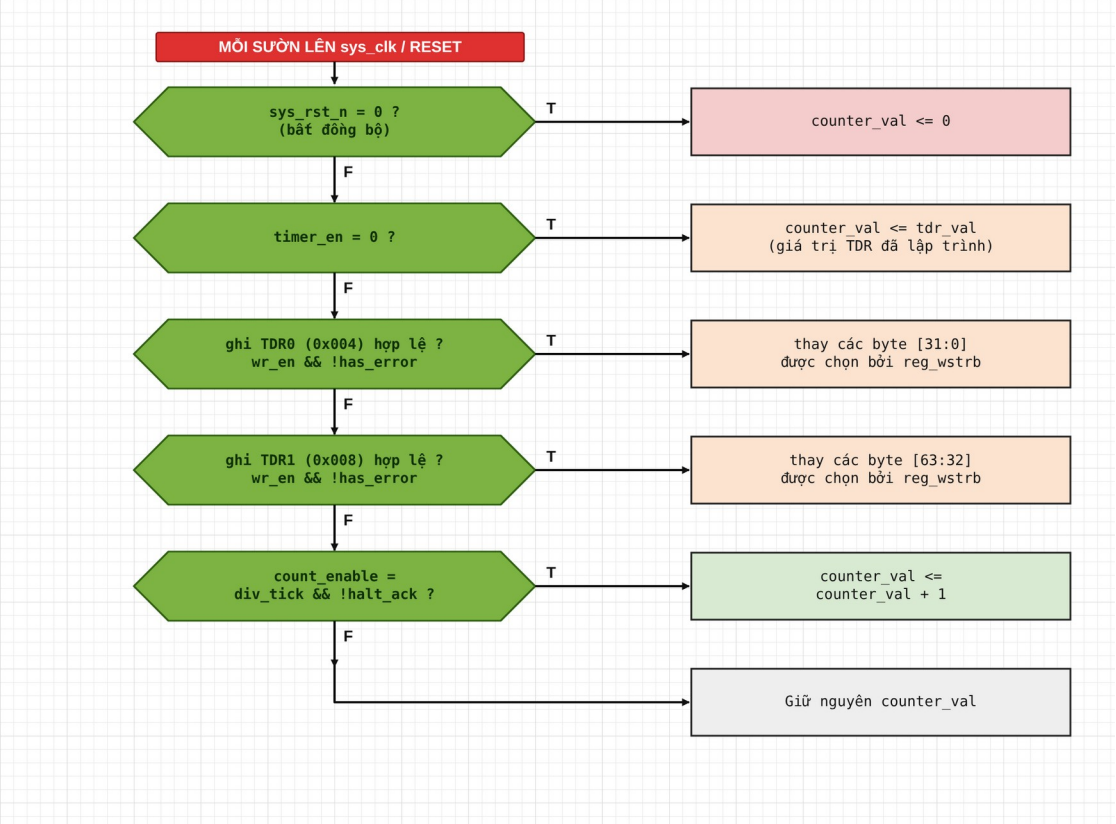
Bảng 3.8: Đặc tả khối counter_core.

3.5.2. **Thứ tự ưu tiên cập nhật bộ đếm 64-bit**

Giá trị bộ đếm được cập nhật tại sườn lên của sys_clk theo thứ tự ưu tiên giảm dần (Hình 3.14, Hình 3.15):

- **Reset:** sys_rst_n = 0 đưa bộ đếm về 0 ngay lập tức (bất đồng bộ).

- **Timer tắt (timer_en = 0):** bộ đếm lấy giá trị tdr_val – giá trị TDR mà phần mềm đã lập trình, nhờ đó phần mềm có thể đặt giá trị bắt đầu trước khi bật timer.
- **Ghi TDR khi timer đang chạy:** lệnh ghi hợp lệ vào địa chỉ 0x004/0x008 thay các byte được chọn bởi reg_wstrb của nửa thấp/nửa cao.
- **Đếm lên:** khi count_enable = div_tick && !halt_ack, bộ đếm tăng 1 với phép cộng 64-bit đầy đủ; nếu không, bộ đếm giữ nguyên.



Hình 3.14: Lưu đồ thứ tự ưu tiên cập nhật bộ đếm của khối counter_core.

counter.v

```
// counter_core: cap nhat bo dem 64-bit theo thu tu uu tien
assign cmp_match_evt = (counter_val == tcmp_val);
assign count_enable  = div_tick && !halt_ack;
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n)      counter_val <= 64'd0;
    else if (!timer_en)  counter_val <= tdr_val;   // theo TDR da lap trinh
    else if (wr_en && !has_error && (reg_addr == 12'h004))
        ...  // thay cac byte [31:0] duoc chon boi reg_wstrb
    else if (wr_en && !has_error && (reg_addr == 12'h008))
        ...  // thay cac byte [63:32] duoc chon boi reg_wstrb
    else if (count_enable)
        counter_val <= counter_val + 64'd1;
end
```

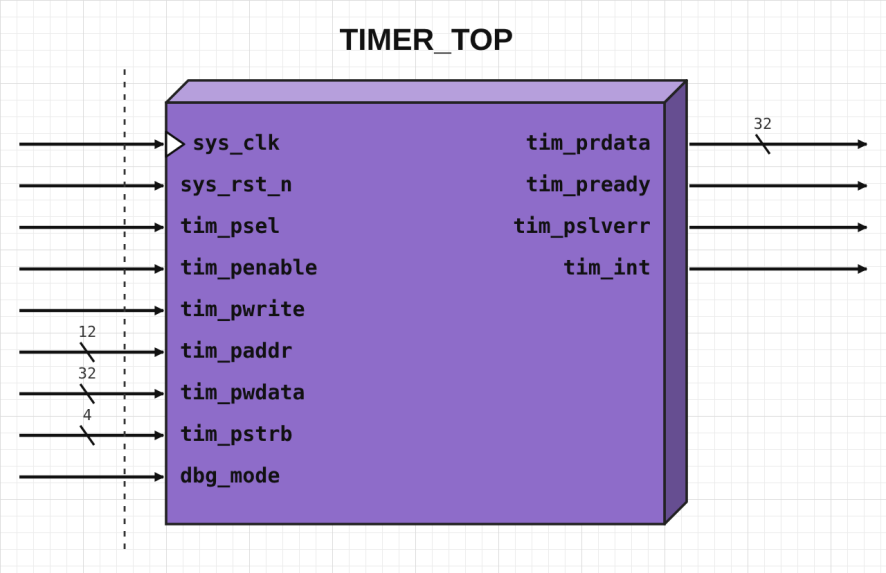
Hình 3.15: Đoạn code cập nhật bộ đếm 64-bit của counter_core.

3.5.3. **Bộ so sánh và tạo sự kiện ngắt**

Bộ so sánh tạo `cmp_match_evt = (counter_val == tcmp_val)` dưới dạng mức logic tổ hợp 64-bit. Tín hiệu này được đưa về timer_register làm int_set để bật cờ int_st; vì cờ được chốt, ngắt vẫn được giữ cho tới khi phần mềm xóa bằng RW1C dù bộ đếm đã tiếp tục tăng. Giá trị reset của TCMP là 0xFFFF_FFFF_FFFF_FFFF nên sau reset không phát sinh ngắt ngoài ý muốn.

3.6. **Module TOP (timer_top) và cơ chế ghép nối các khối**

Khối timer_top đóng vai trò là “bo mạch chủ” kết nối các module con bằng các dây có hậu tố _w (reg_addr_w, timer_en_w, tcmp_w, int_set_w, halt_ack_w…). Giao diện bên ngoài của IP gồm cổng clock/reset, bus APB, tín hiệu dbg_mode và ngõ ra ngắt tim_int (Hình 3.16, Bảng 3.9).



Hình 3.16: Khối TIMER_TOP.

Tên cổng	Chiều	Độ rộng	Chức năng
sys_clk	input	1 bit	Xung nhịp hệ thống (System clock).
sys_rst_n	input	1 bit	Reset bất đồng bộ toàn IP, tích cực mức thấp.
tim_psel, tim_penable, tim_pwrite	input	1 bit	Tín hiệu điều khiển giao dịch APB.
tim_paddr	input	12 bits	Địa chỉ thanh ghi (offset byte).
tim_pwdata / tim_pstrb	input	32 / 4 bits	Dữ liệu ghi và byte strobe.
tim_prdata	output	32 bits	Dữ liệu đọc.
tim_pready	output	1 bit	Slave hoàn tất giao dịch (sau 1 chu kỳ chờ).
tim_pslverr	output	1 bit	Báo lỗi lệnh ghi TCR bị từ chối.

Tên cổng	Chiều	Độ rộng	Chức năng
dbg_mode	input	1 bit	Hệ thống đang ở chế độ gỡ lỗi.
tim_int	output	1 bit	Ngắt gửi tới CPU (int_st & int_en).

Bảng 3.9: Đặc tả khối timer_top.

Bên cạnh việc nối dây, timer_top chứa hai phần logic nhỏ nhưng quan trọng (Hình 3.17):

- `assign reg_wr_en_w = reg_valid_w && reg_write_w;` – chỉ khi giao dịch APB hoàn tất và là lệnh ghi thì khối thanh ghi và lõi đếm mới được phép thay đổi trạng thái.
- **Thanh ghi tdr_shadow 64-bit:** cập nhật mỗi khi phần mềm ghi TDR0/TDR1 (xung tdr0_load/tdr1_load). Khi timer tắt, counter_core liên tục lấy giá trị này, nhờ đó giá trị TDR đọc lại luôn đúng với giá trị phần mềm đã lập trình.

timer_top.v

```
// timer_top: lenh ghi da hoan tat + thanh ghi tdr_shadow
assign reg_wr_en_w = reg_valid_w && reg_write_w;
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n) tdr_shadow <= 64'd0;
    else begin
        if (tdr0_load_w) tdr_shadow[31:0]  <= tdr0_wdata_w;
        if (tdr1_load_w) tdr_shadow[63:32] <= tdr1_wdata_w;
    end
end
```

Hình 3.17: Đoạn code tạo reg_wr_en_w và thanh ghi tdr_shadow trong timer_top.

Testbench còn quan sát trực tiếp các tín hiệu nội bộ dut.u_counter.counter_val, dut.u_counter.u_counter_divider.div_tick và dut.u_counter.halt_ack bằng tham chiếu phân cấp để kiểm tra bộ đếm, bộ chia tần và chế độ dừng ở mức hệ thống.

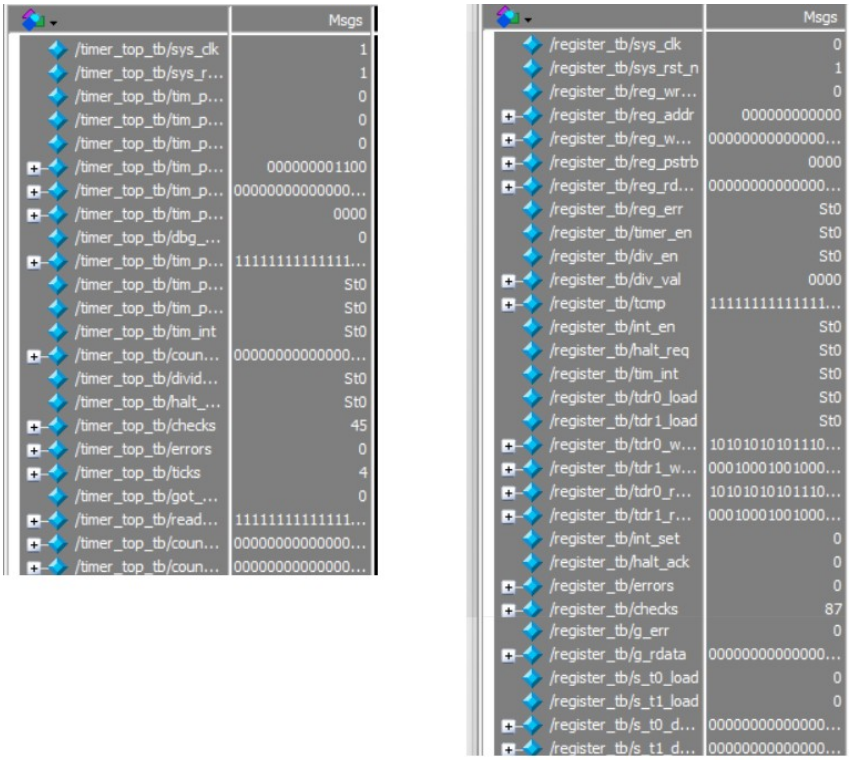
Chương 4. **KẾT QUẢ KIỂM THỬ VÀ MÔ PHỎNG HỆ THỐNG**

4.1. **Môi trường kiểm thử tự động (Self-checking Testbench)**

Để khẳng định lõi IP Timer hoạt động đúng mà không cần phải "căng mắt" ngồi dò từng đường sóng (Waveform) thủ công, nhóm đã xây dựng các môi trường kiểm thử tự động: `timer_top_tb` cho toàn hệ thống và `register_tb`, `counter_tb`, `app_slave_tb` cho từng khối chức năng. Môi trường này hoạt động như một vị trọng tài, tự động cấp dữ liệu và tự động chấm điểm đúng/sai.

Nguyên lý hoạt động cơ bản:

- Tạo "Đáp án chuẩn" (Golden Model): Mô hình tham chiếu `GOLDEN_MODEL.py` của đề bài được viết lại thành module Verilog `counter_golden_model` (`GOLDEN_MODEL.v`). Mô hình này chạy song song với khối counter và tính sẵn kết quả kỳ vọng ở từng chu kỳ; với các khối còn lại, giá trị kỳ vọng được lấy trực tiếp từ đặc tả (giá trị reset, bản đồ thanh ghi, quy tắc lỗi TCR, cơ chế RW1C).
- Đóng giả CPU (Stimulus): Testbench tạo xung nhịp sys_clk 100 MHz, giữ reset ở các chu kỳ đầu rồi đóng vai CPU, phát các giao dịch APB (task `apb_write`, `apb_read`) theo đúng chuẩn SETUP – ACCESS vào bus của IP Timer.
- Tự động thu thập và chấm điểm (Monitor & Scoreboard): Testbench liên tục giám sát dữ liệu đọc về, tín hiệu PSLVERR, ngắt tim_int, xung div_tick và giá trị bộ đếm. Mỗi lần so sánh, biến `checks` tăng 1; nếu sai lệch dù chỉ 1 bit, biến `errors` tăng 1 và thời điểm lỗi được in ra. Kết quả cuối cùng hiển thị ở cột Msgs của cửa sổ Wave (Hình 4.1).

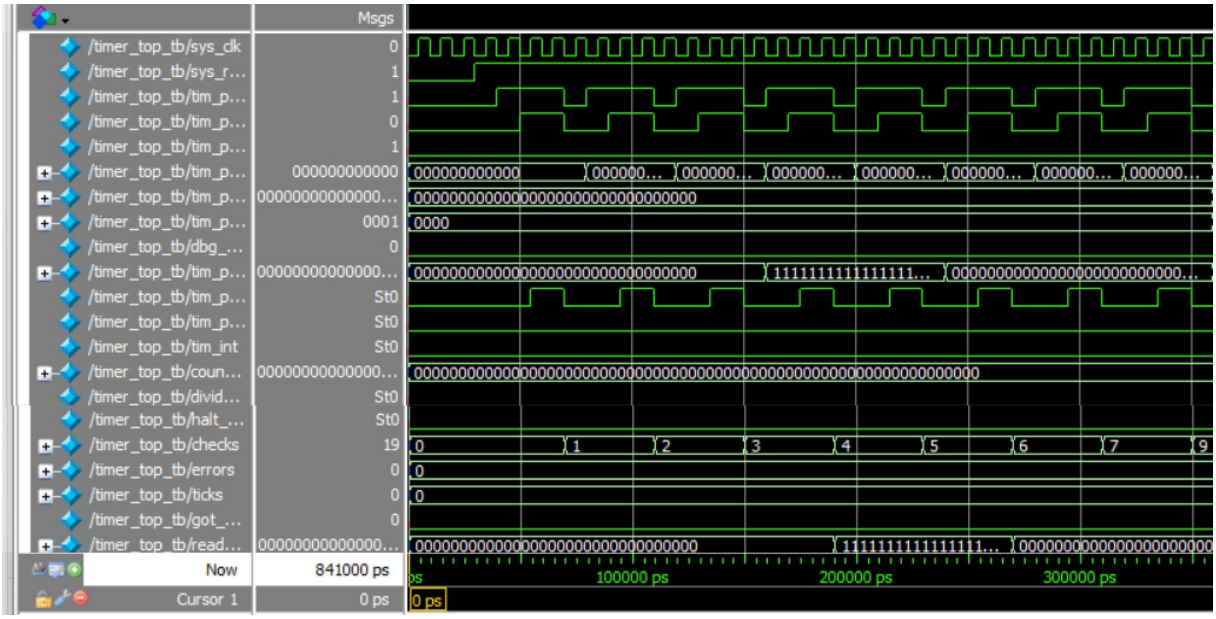


(a) timer_top_tb **(b) register_tb**

Hình 4.1: Báo cáo kết quả kiểm tra tự động (Test Summary) của khối TIMER_TOP và TIMER_REGISTER.

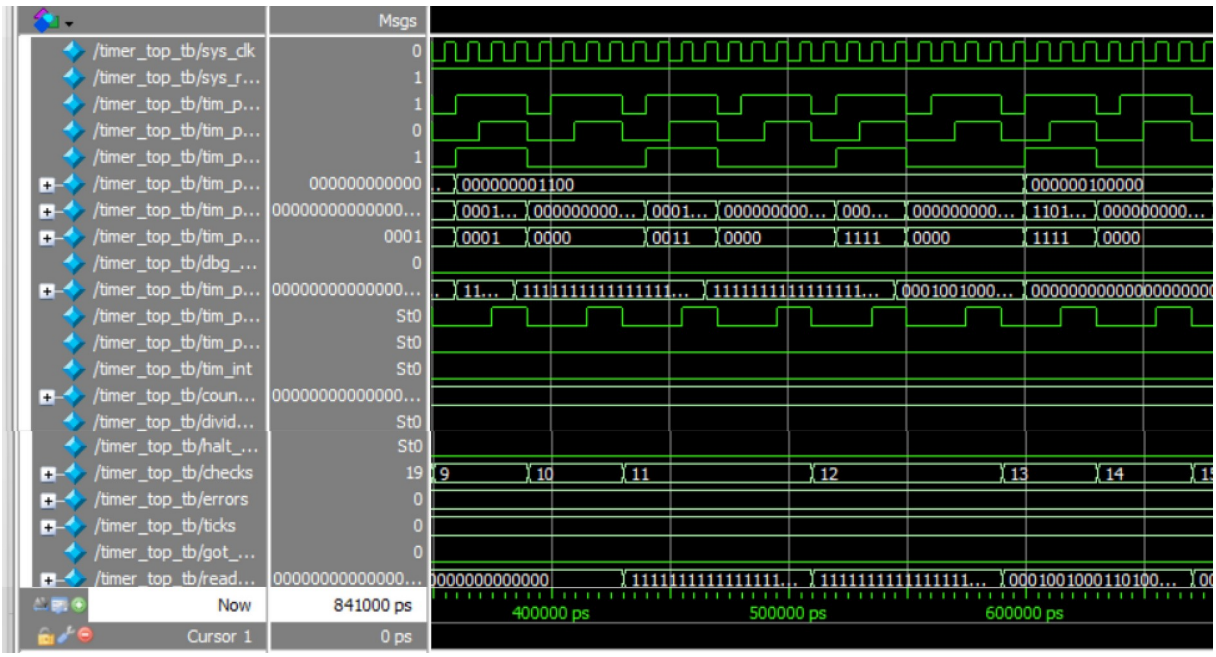
4.2. **Kết quả mô phỏng trên phần mềm ModelSim**

4.2.1. **Kịch bản kiểm tra giá trị reset của các thanh ghi (TEST 1)**



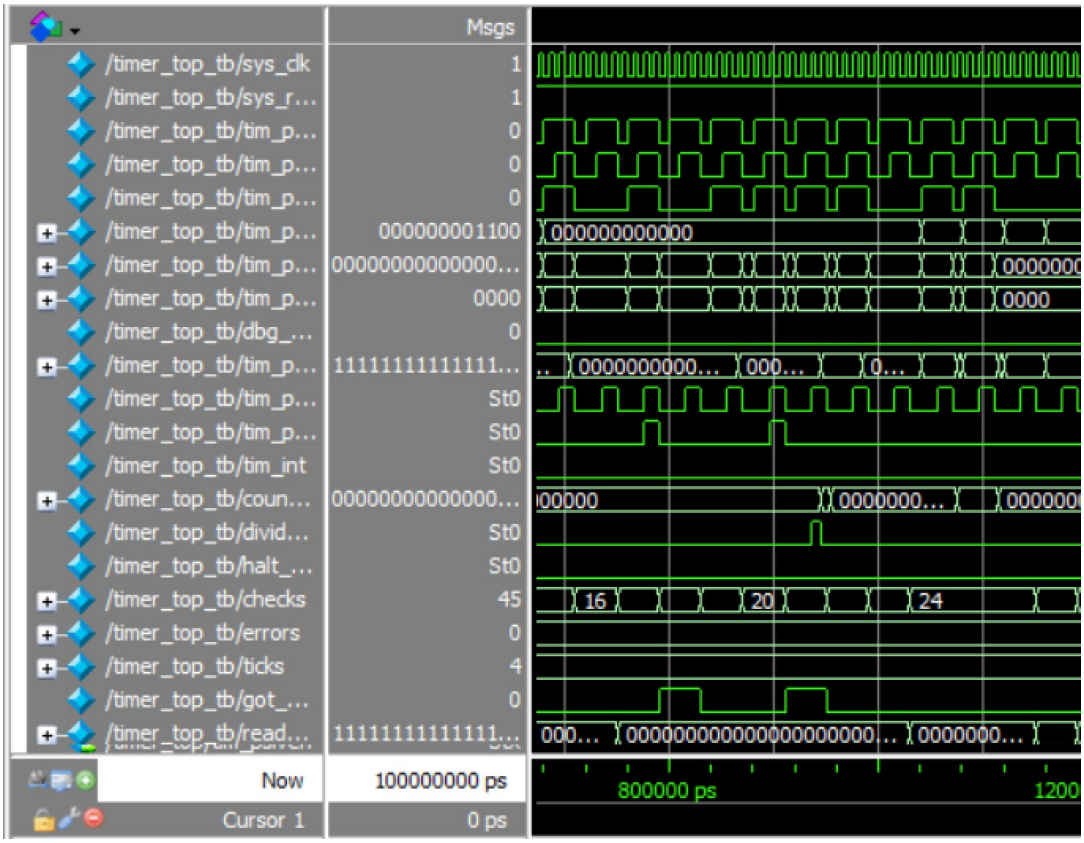
Hình 4.2: Mô phỏng Waveform đọc giá trị reset của 8 thanh ghi qua bus APB.

4.2.2. **Kịch bản truy cập theo byte strobe và địa chỉ dự trữ (TEST 2)**



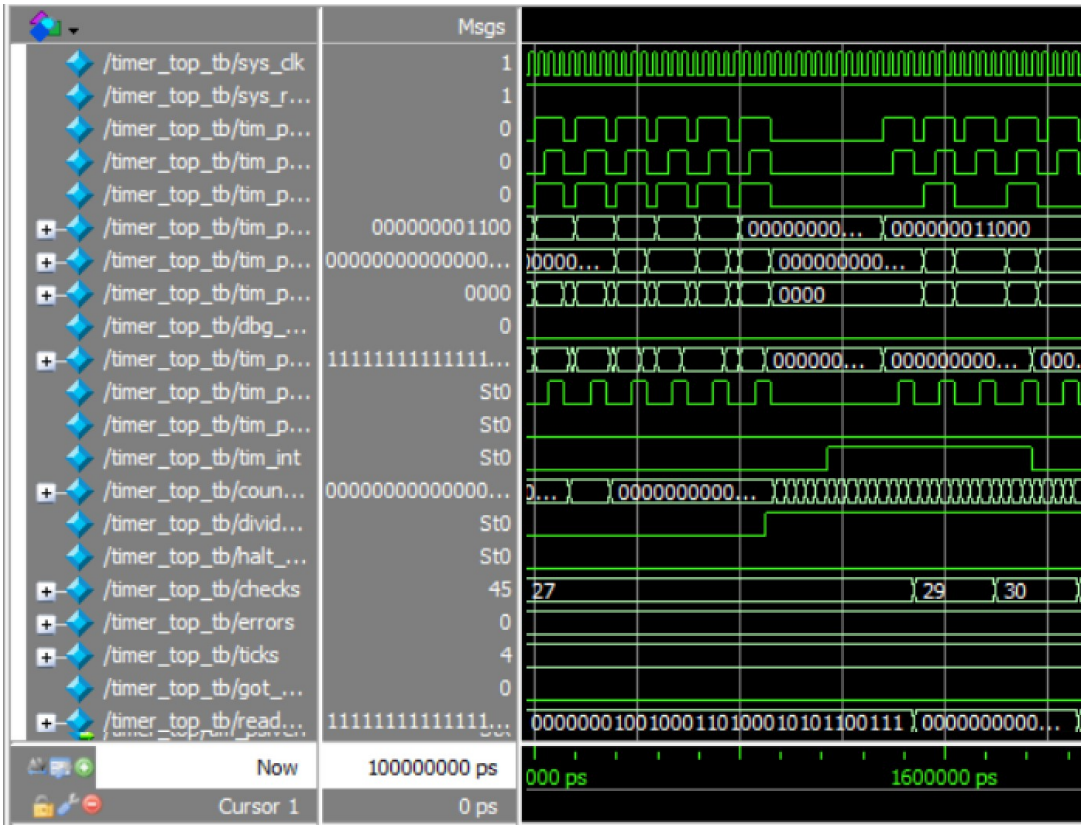
Hình 4.3: Mô phỏng Waveform ghi thanh ghi TCMP0 với PSTRB = 0001, 0011, 1111.

4.2.3. **Kịch bản cấu hình bộ chia tần và bộ đếm 64-bit (TEST 3 – TEST 4)**



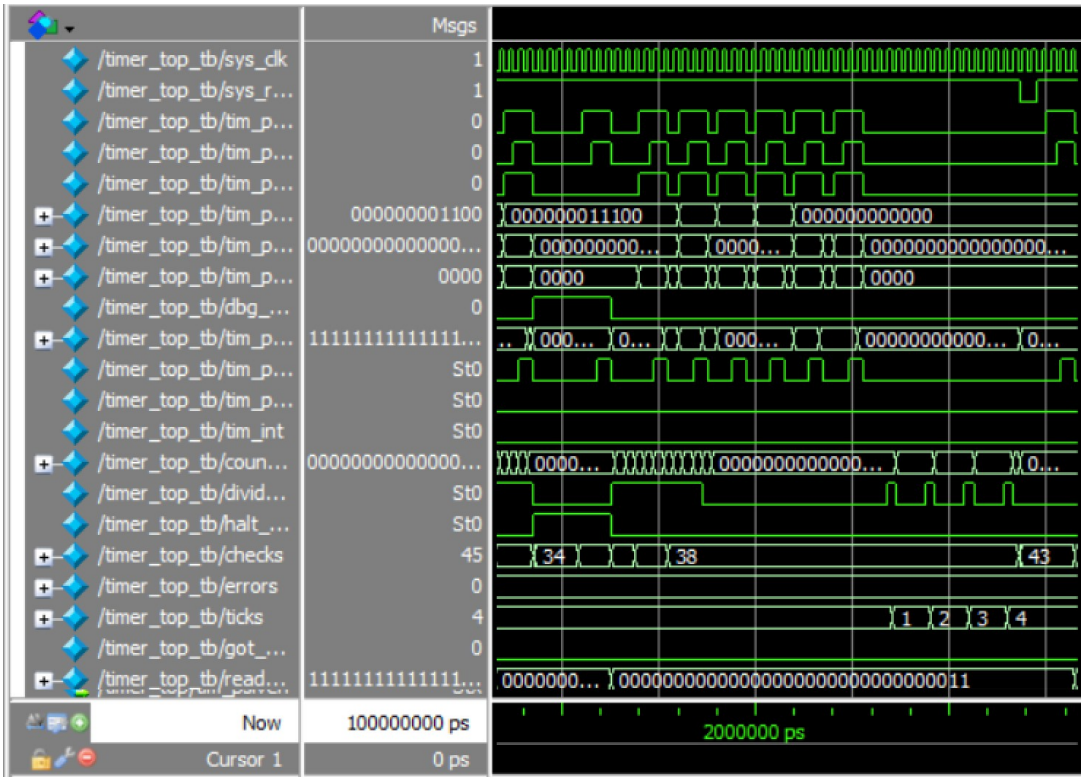
Hình 4.4: Mô phỏng Waveform tín hiệu PSLVERR khi ghi TCR không hợp lệ và ghi TDR0/TDR1.

4.2.4. **Kịch bản ngắt so sánh và xóa cờ RW1C (TEST 5)**



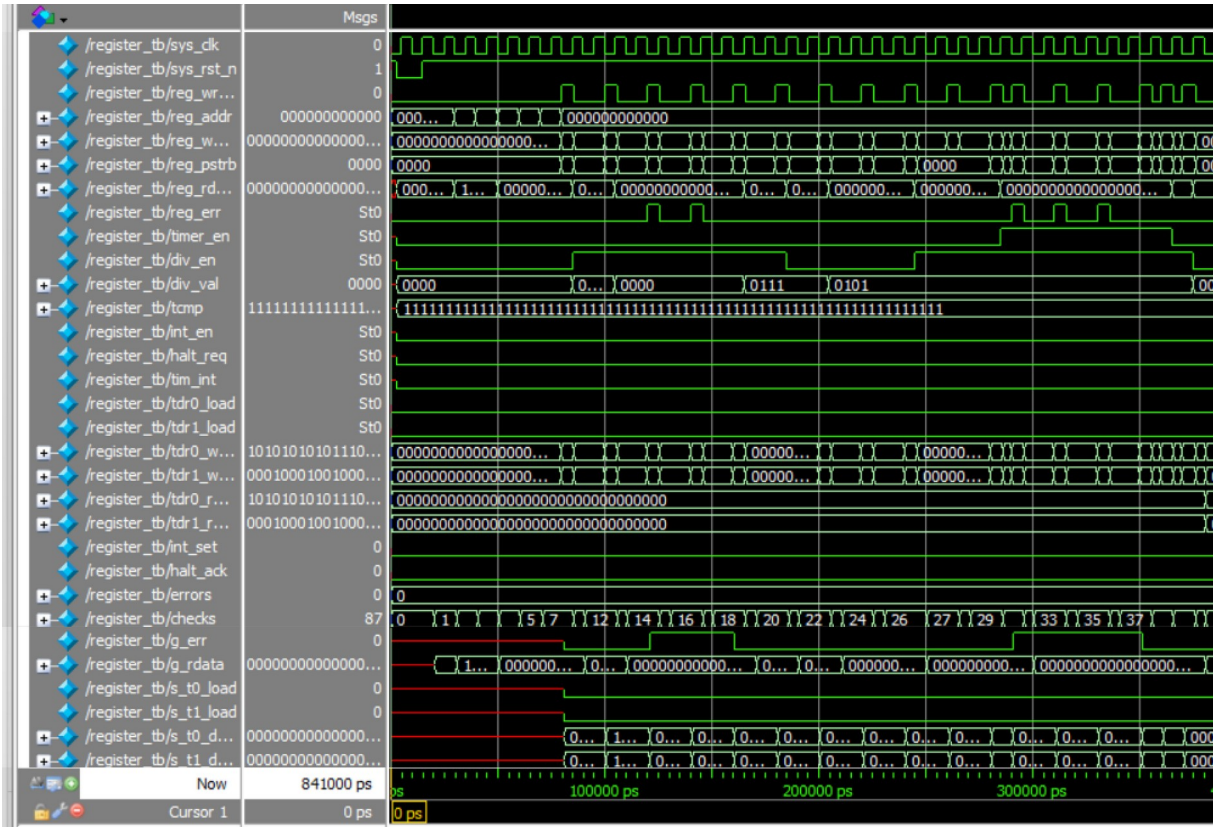
Hình 4.5: Mô phỏng Waveform tín hiệu tim_int khi bộ đếm bằng TCMP và khi xóa cờ bằng W1C.

4.2.5. **Kịch bản dừng gỡ lỗi, chia tần và reset bất đồng bộ (TEST 6 – TEST 8)**

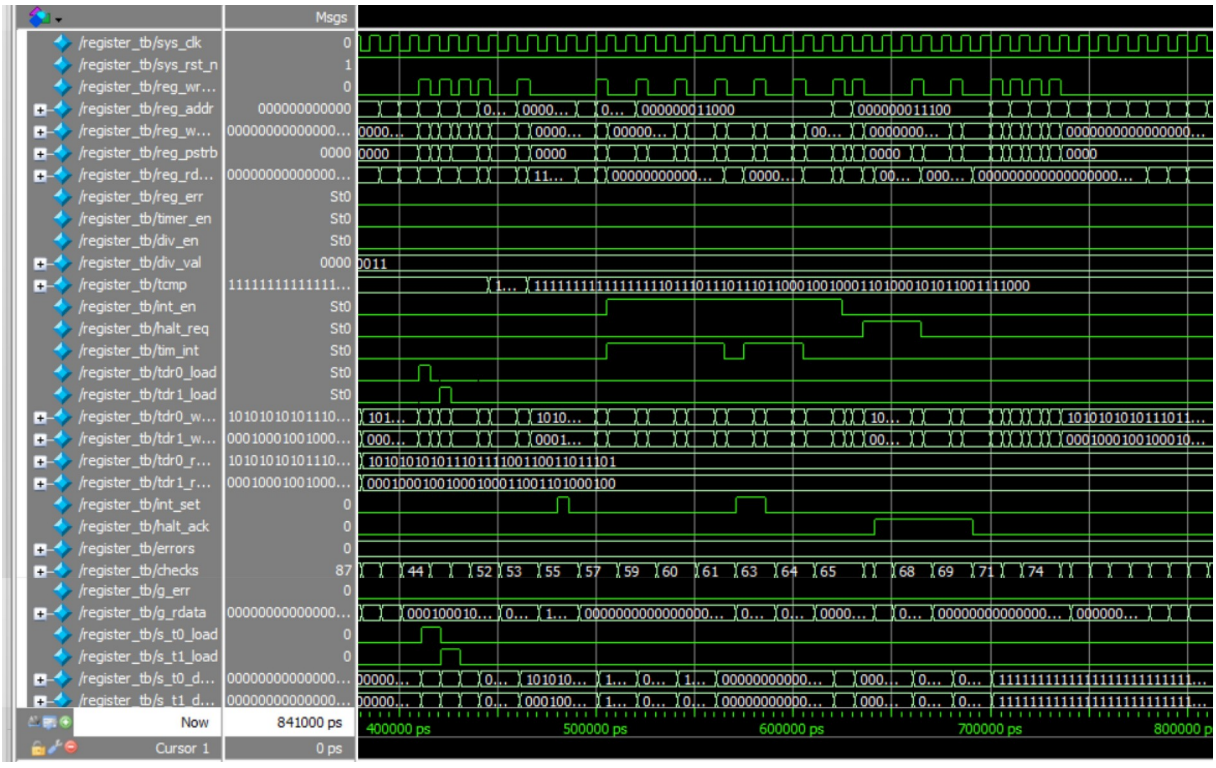


Hình 4.6: Mô phỏng Waveform tín hiệu halt_ack, divider_tick và reset bất đồng bộ.

4.2.6. **Kịch bản kiểm thử khối thanh ghi (register_tb)**

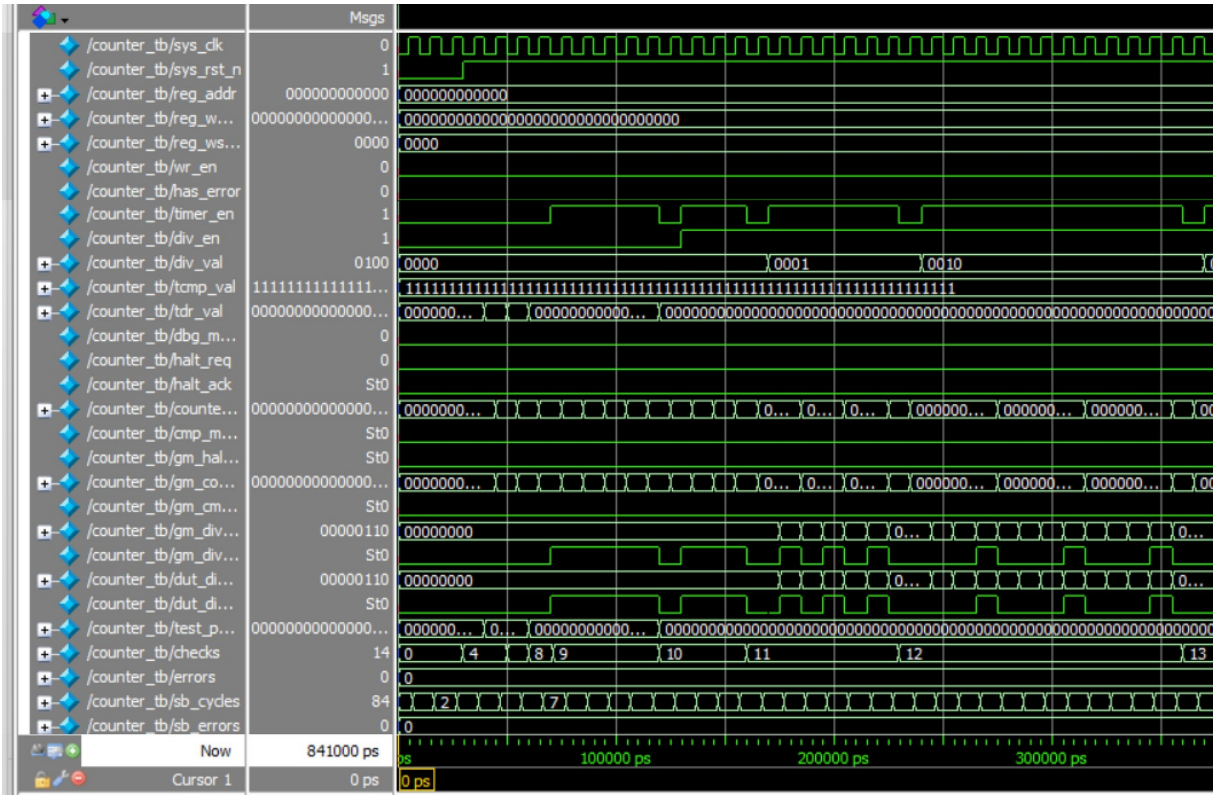


Hình 4.7: Mô phỏng Waveform khối thanh ghi T1 – T6: cấu hình TCR và tín hiệu reg_err.

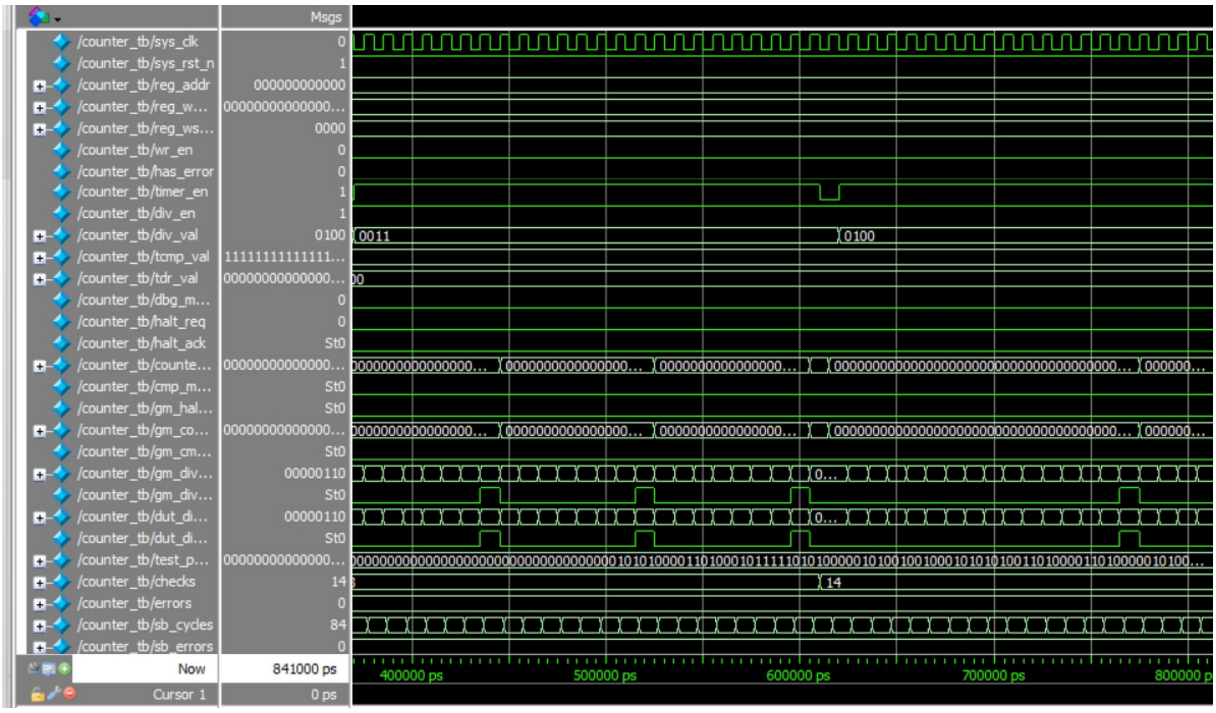


Hình 4.8: Mô phỏng Waveform khối thanh ghi T7 – T12: TDR, cờ ngắt TISR và THCSR.

4.2.7. **Kịch bản đối chiếu bộ đếm với Golden Model (counter_tb)**



Hình 4.9: Mô phỏng Waveform so sánh counter.v với GOLDEN_MODEL.v (div_val = 0 – 2).



Hình 4.10: Mô phỏng Waveform so sánh counter.v với GOLDEN_MODEL.v (div_val = 3 – 4).

Chương 5. **KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN CỦA ĐỀ TÀI**

5.1. **Kết luận**

Nhìn lại toàn bộ quá trình thực hiện đồ án, nhóm đã xây dựng thành công một lõi IP Timer 64-bit hoàn chỉnh giao tiếp AMBA APB từ đặc tả tới mô phỏng. Cụ thể, đề tài đã đạt được những kết quả thực tế như sau:

- **Hoàn thiện giao tiếp APB chuẩn:** apb_slave chèn đúng 1 chu kỳ chờ, chỉ phát yêu cầu ghi một lần ở chu kỳ hoàn tất và chỉ báo PSLVERR cho lệnh ghi bị từ chối.
- **Bộ đếm 64-bit với bộ chia tần an toàn:** Bộ chia tần dạng xung cho phép đếm giữ toàn bộ IP trong một miền clock, có bảo vệ cấu hình TCR khi timer đang chạy và từ chối div_val 8..15.
- **Ngắt và gỡ lỗi đúng đặc tả:** Cờ ngắt chốt với cơ chế RW1C ưu tiên sự kiện phần cứng, che ngắt bằng TIER, chế độ Debug Halt đóng băng cả bộ chia tần lẫn bộ đếm.
- **Tự động hóa khâu kiểm tra:** Bốn testbench tự kiểm tra cùng Scoreboard đối chiếu Golden Model mỗi chu kỳ; trên ModelSim, testbench hệ thống đạt 45/45 và khối thanh ghi đạt 87/87 điểm kiểm tra, không có lỗi, dạng sóng xác nhận đúng giao thức APB và chức năng của IP.

5.2. **Những khó khăn trong quá trình thiết kế**

Golden Model viết bằng Python không chạy được trên ModelSim: Mô hình tham chiếu GOLDEN_MODEL.py của đề bài là mã Python, trong khi vlog của ModelSim chỉ biên dịch Verilog/SystemVerilog nên tệp này báo lỗi khi đưa vào project. Nhóm khắc phục bằng cách viết lại mô hình thành module Verilog counter_golden_model (GOLDEN_MODEL.v) theo đúng cấu trúc step_clk() / eval_combinational() của bản Python và xây dựng counter_tb để so sánh RTL với mô hình ở mọi chu kỳ.

Lỗi “Startup(-L) … Load canceled” khi nạp mô phỏng: Khi tích hợp, module timer_top khởi tạo timer_register trong khi tệp register.v ban đầu khai báo module tên register; một số tệp testbench trong project còn rỗng. ModelSim vẫn báo biên dịch thành công nhưng bước nạp (elaboration) thất bại, và hộp thoại Start Simulation chỉ hiển thị thông báo chung “Load canceled”, che mất lỗi thật. Nhóm thống nhất tên module timer_register, bổ sung đầy đủ mã nguồn và viết các script run_*.do kiểm tra tệp rỗng, biên dịch và chạy vsim trực tiếp để Transcript hiển thị đúng thông báo lỗi.

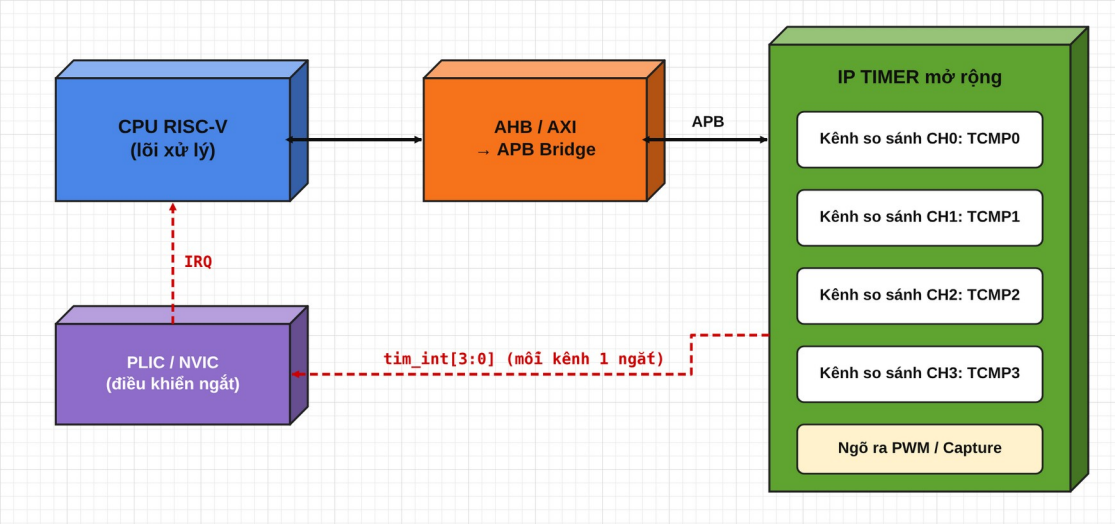
Không thống nhất độ rộng bus giữa các khối: Phiên bản apb_slave ban đầu dùng địa chỉ 32-bit/8-bit trong khi timer_top dùng bus địa chỉ 12-bit, khiến các bit địa chỉ cao bị thả nổi (z), đồng thời PSLVERR được bật cả với lệnh đọc. Nhóm thống nhất giao

diện 12-bit cho toàn IP, chỉ cho phép PSLVERR ở lệnh ghi và bổ sung app_slave_tb để khóa lại hành vi này.

Tranh chấp thời điểm lấy mẫu trong testbench (race condition): Một số điểm kiểm tra ban đầu đọc kết quả đúng tại sườn lên của clock nên nhận giá trị cũ hoặc mới tùy thứ tự sự kiện của trình mô phỏng. Nhóm áp dụng quy ước thay đổi kích thích ở sườn xuống, DUT lấy mẫu ở sườn lên và Scoreboard so sánh sau sườn lên 2 ns, nhờ đó kết quả mô phỏng ổn định trên mọi trình mô phỏng.

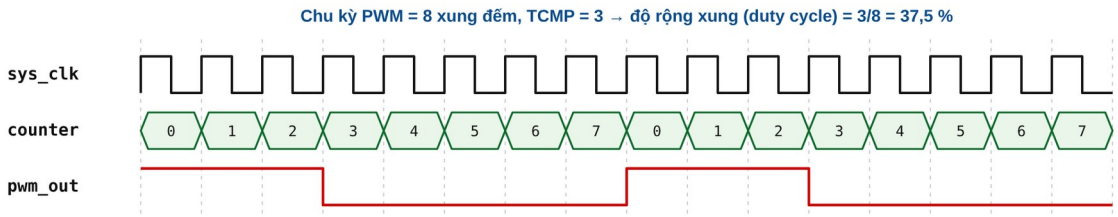
5.3. **Hướng phát triển trong tương lai**

- **Mở rộng nhiều kênh so sánh:** Bổ sung nhiều thanh ghi TCMP (CH0 – CH3), mỗi kênh một cờ ngắt riêng, kết nối với bộ điều khiển ngắt (PLIC/NVIC) của CPU RISC-V hoặc ARM để phục vụ nhiều tác vụ định thời cùng lúc.



Hình 5.1: Ví dụ IP Timer nhiều kênh so sánh kết nối với CPU RISC-V.

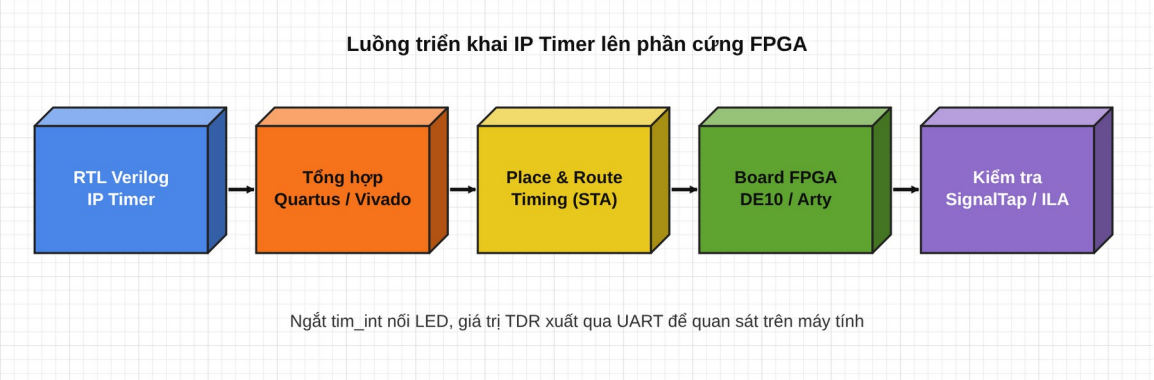
- **Thêm chức năng PWM và Input Capture:** Dùng bộ đếm và giá trị so sánh để tạo xung điều chế độ rộng (PWM) cho điều khiển động cơ, LED; hoặc chụp giá trị bộ đếm khi có sự kiện ngoài để đo độ rộng xung.



Hình 5.2: Ví dụ tạo xung PWM từ bộ đếm và giá trị so sánh.

- **Đưa xuống chạy thử trên "đồ thật":** Tổng hợp thiết kế bằng Quartus/Vivado, nạp xuống board FPGA, kiểm tra định thời tĩnh (STA) và

quan sát tín hiệu thực bằng SignalTap/ILA; ngắt tim_int có thể nối ra LED và giá trị bộ đếm gửi lên máy tính qua UART.



Hình 5.3: Ví dụ luồng triển khai IP Timer lên board FPGA.

TÀI LIỆU THAM KHẢO

[1] ARM Limited, “AMBA APB Protocol Specification, Version 2.0”, ARM IHI 0024C, 2010.

[2] IEEE Computer Society, “IEEE Standard for Verilog Hardware Description Language”, IEEE Std 1364-2005, 2006.

[3] Tài liệu đặc tả thiết kế IP Timer (Design Specification) và mô hình tham chiếu GOLDEN_MODEL.py, học phần FAP201, Trường Đại học FPT.

[4] Siemens EDA, “ModelSim User’s Manual” và “ModelSim Command Reference Manual”.

[5] Nhóm Trương Vũ (lớp IC2007), “Thiết kế và Mô phỏng giao thức truyền thông UART-FIFO”, báo cáo đồ án FAP201 (tham khảo cách trình bày).